

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
DEPARTMENT OF MATHEMATICS**



GRADUATION THESIS

**Explainable Deep Reinforcement Learning for Multi-Asset
Trading via Transformer-Based State Encoding**

Submitted in partial fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE
IN APPLIED MATHEMATICS
SPECIALIZATION IN FINANCIAL ENGINEERING
AND RISK MANAGEMENT**

Student's Name: LE NHAT QUYEN

Student's ID: MAMAIU21100

Thesis Supervisor: ASSOC. PROF. NGUYEN MINH QUAN

Ho Chi Minh City, Vietnam

June 2026

Explainable Deep Reinforcement Learning for Multi-Asset Trading via Transformer-Based State Encoding

By

LE NHAT QUYEN

Submitted to DEPARTMENT OF MATHEMATICS,
INTERNATIONAL UNIVERSITY, HO CHI MINH CITY
in partial fulfillment of requirements for the degree of
BACHELOR OF SCIENCE
IN APPLIED MATHEMATICS
SPECIALIZATION IN FINANCIAL ENGINEERING
AND RISK MANAGEMENT

June 2026

Signature of Student: _____



LE NHAT QUYEN

Certified by: _____



ASSOC. PROF. NGUYEN MINH QUAN

Thesis Supervisor

Approved by: _____

ASSOC. PROF. MAI DUC THANH
ACTING HEAD OF DEPARTMENT OF MATHEMATICS

Explainable Deep Reinforcement Learning for Multi-Asset Trading via Transformer-Based State Encoding

By

LE NHAT QUYEN

Submitted to DEPARTMENT OF MATHEMATICS,
INTERNATIONAL UNIVERSITY, HO CHI MINH CITY
in partial fulfillment of requirements for the degree of
BACHELOR OF SCIENCE
IN APPLIED MATHEMATICS
SPECIALIZATION IN FINANCIAL ENGINEERING
AND RISK MANAGEMENT

ABSTRACT

Portfolio allocation is a sequential decision-making problem because portfolio weights must be updated as market conditions change. This thesis proposes an explainable deep reinforcement learning framework for multi-asset trading by combining a Transformer-based state encoder with Proximal Policy Optimization (PPO). The Transformer encodes historical multi-asset data into latent market states, while an actor-critic policy generates dynamic portfolio weights in a long-only, fully invested setting with transaction costs incorporated into the reward function. The empirical study compares the Transformer encoder with MLP-based and LSTM-based baselines in a controlled five-asset equity universe. The results show that the Transformer-based policy achieves the strongest performance among the learned models in the main setting, while simple passive benchmarks remain strong in the full benchmark comparison. The thesis also provides an attention-based temporal analysis of the trained Transformer encoder to support the explainability objective.

Keywords: Deep Reinforcement Learning, Transformer, Proximal Policy Optimization, Dynamic Portfolio Allocation, Multi-Asset Trading, Transaction Costs, Attention-Based Interpretability.

ACKNOWLEDGEMENT

I would like to thank my thesis supervisor, Assoc. Prof. Dr. Nguyen Minh Quan, for guiding me during the development of this thesis. His comments helped me clarify the research direction, improve the methodology, and interpret the experimental results more carefully. I am especially grateful for his patience and support when the implementation and research design had to be revised.

I would also like to thank the Department of Mathematics, International University, Vietnam National University - Ho Chi Minh City, for providing the academic environment in which I could complete this work. The courses in the Applied Mathematics major gave me the background needed to study portfolio optimization, quantitative finance, deep learning, and reinforcement learning.

I am deeply thankful to my family for their trust and support throughout my university years. Their encouragement helped me continue working through difficult periods of study and research. I would also like to thank my friends for their companionship and for making this journey less stressful.

Although I have tried to complete this thesis carefully, shortcomings may remain. I sincerely welcome comments and suggestions that can help improve this work.

Contents

ABSTRACT	1
ACKNOWLEDGEMENT	2
List of Abbreviations	2
1 INTRODUCTION	3
1.1 Background and Motivation	3
1.2 Limitations of existing approaches	5
1.3 Research problem	6
1.4 Research objectives	7
1.5 Research questions	8
1.6 Overview of the proposed approach	9
1.7 Scope and constraints	10
1.8 Thesis structure	10
2 LITERATURE REVIEW	11
2.1 Introduction	11
2.2 Portfolio optimization and dynamic allocation	11
2.3 Machine learning and deep learning in finance	12
2.3.1 Prediction-oriented financial modeling	13
2.3.2 Sequence modeling approaches in finance	13
2.4 Attention mechanisms and Transformers	14
2.4.1 Attention fundamentals	14
2.4.2 Transformer encoder architecture	14

2.4.3	Transformer as a market representation learner	15
2.5	Reinforcement learning for portfolio management	16
2.5.1	Reinforcement learning as a sequential decision framework . .	16
2.5.2	Deep Reinforcement Learning in Portfolio Management	16
2.5.3	PPO for policy optimization	17
2.6	Transaction costs and practical reward design	18
2.7	Literature Synthesis and Research Gap	19
2.8	Chapter Summary	20
3	PROPOSED METHODOLOGY	21
3.1	Chapter Overview	21
3.2	Problem Formulation	22
3.2.1	Dynamic Portfolio Allocation	22
3.2.2	Markov Decision Process Formulation	23
3.3	State Representation	24
3.4	Assumptions, Constraints, and Notation	25
3.5	Transformer-Based Market Representation Encoder	26
3.6	Attention-Based Interpretability	29
3.7	Actor–Critic Architecture and Portfolio Action	30
3.8	Portfolio Return and Transaction Cost Modeling	31
3.9	Reward Function	33
3.10	PPO-Based Policy Optimization	34
3.11	Main Framework of the Proposed Methodology	37
3.12	Suitability to Research Questions	40
3.13	Chapter Summary	40
4	EXPERIMENTAL DESIGN AND IMPLEMENTATION	42
4.1	Experimental Objectives	42
4.2	Main Dataset Description	43
4.3	Data Preparation and State Construction	44
4.4	Portfolio Constraints and Trading Assumptions	45
4.5	Benchmark and Model Configurations	46

4.5.1	Benchmark Strategies	46
4.5.2	Learned Encoder Variants	47
4.6	Training, Validation, and Test Protocol	49
4.7	Reward Design and Transaction Cost Modeling	51
4.8	Evaluation Metrics	53
4.9	Attention-Based Explainability Protocol	55
4.10	Reliability and Fairness of Evaluation	56
4.11	Chapter Summary	57
5	EMPIRICAL RESULTS AND DISCUSSION	59
5.1	Chapter Overview	59
5.2	Overall Out-of-Sample Portfolio Performance	60
5.2.1	Main Test-Set Performance Comparison	60
5.2.2	Portfolio Value Dynamics	61
5.2.3	Practical Interpretation of Benchmark Results	63
5.3	Effect of Market Representation on Learned Policies	64
5.3.1	Learned Encoder Comparison under PPO	64
5.3.2	Transformer-Based Representation versus MLP and LSTM Encoders	66
5.3.3	Risk and Turnover Profile of Learned Models	67
5.4	Effect of Transaction-Cost-Aware Reward Design	68
5.4.1	Cost-Aware and No-Cost Transformer Variants	69
5.4.2	Return–Risk–Turnover Trade-off	71
5.4.3	Portfolio Allocation Behavior under Different Reward Designs	73
5.5	Temporal Attention and Interpretability Analysis	74
5.5.1	Representative Attention Heatmaps	75
5.5.2	Grouped Attention Patterns under Different Market Conditions	76
5.5.3	Interpretation Scope and Statistical Support	78
5.6	Lookback Window Setting	78
5.7	Integrated Discussion of Findings	79
5.8	Chapter Summary	81

6	CONCLUSION AND FUTURE WORK	82
6.1	Chapter Overview	82
6.2	Summary of the Research	82
6.3	Summary of Main Findings	83
6.4	Limitations	84
6.5	Future Work	85
6.6	Final Remarks	86
	APPENDIX	88
A	SUPPLEMENTARY PERFORMANCE RESULTS	88
A.1	Supplementary Learned-Model Metric Comparison	88
A.2	Supplementary Reward-Design Comparison	89
B	ADDITIONAL TEMPORAL ATTENTION ANALYSIS	91
B.1	Grouped Attention Summary Statistics	91
B.2	Interpretation and Robustness Notes	93
C	EXTENDED-HORIZON ROBUSTNESS RESULTS	94
C.1	Ten-Year Experimental Setting	94
C.2	Supplementary Ten-Year Result Comparison	94
C.3	Interpretation Relative to the Main Five-Year Setting	96
D	IMPLEMENTATION NOTES	97
D.1	Feature Construction Details	97
D.2	Hyperparameter Documentation	97
D.3	Portfolio Environment and Evaluation Notes	99
D.4	Code Repository and Supplementary Materials	99
	Bibliography	100

List of Tables

4.1	Main five-asset universe used in the empirical study	43
4.2	Main implementation settings used in the learned-model experiments	51
5.1	Main out-of-sample performance comparison between benchmark strategies and learned PPO-based allocation policies in the five-year setting.	60
5.2	Performance comparison between cost-aware and no-cost Transformer-PPO variants under realistic and frictionless evaluation settings. . . .	69
B.1	Grouped temporal attention summary statistics across market conditions.	92
B.2	Statistical comparison of temporal attention metrics between high-volatility and low-volatility windows.	93
C.1	Ten-year benchmark and learned-model performance comparison. . .	95
D.1	Main experimental configuration	98

List of Figures

3.1	Transformer-PPO Portfolio Rebalancing Framework	38
5.1	Full out-of-sample portfolio value curves for benchmark strategies and PPO-based learned allocation models over the five-year test period. .	62
5.2	Out-of-sample portfolio value curves for learned PPO-based alloca- tion models over the five-year test period.	65
5.3	Out-of-sample portfolio value curves of cost-aware and no-cost Transformer- PPO variants under realistic transaction-cost-aware evaluation.	71
5.4	Drawdown comparison between cost-aware and no-cost Transformer- PPO variants under realistic transaction-cost-aware evaluation.	72
5.5	Average test portfolio weights of cost-aware and no-cost Transformer- PPO variants under realistic evaluation.	73
5.6	Representative layerwise temporal attention heatmaps from the Transformer- PPO model over the five-year test period.	75
5.7	Grouped temporal attention analysis of the Transformer-PPO model across random, high-volatility, and low-volatility market windows. . .	77
A.1	Supplementary metric comparison of learned encoder variants in the five-year setting.	89
A.2	Supplementary reward-design metric comparison for Transformer vari- ants.	90
C.1	Extended-horizon portfolio value curves under the ten-year setting. .	96
D.1	QR code linking to the code repository and supplementary materials	99

List of Abbreviations

AAPL Apple Inc. (ticker)

AMZN Amazon.com, Inc. (ticker)

DRL Deep Reinforcement Learning

FFN Feed-Forward Network

GAE Generalized Advantage Estimation

JNJ Johnson & Johnson (ticker)

JPM JPMorgan Chase & Co. (ticker)

LSTM Long Short-Term Memory

MDD Maximum Drawdown

MDP Markov Decision Process

MLP Multi-Layer Perceptron

MSFT Microsoft Corporation (ticker)

MVO Mean–Variance Optimization

PPO Proximal Policy Optimization

RL Reinforcement Learning

Chapter 1

INTRODUCTION

1.1 Background and Motivation

Portfolio allocation has long been a crucial problem in finance, as investors do not make decisions for each asset in isolation. What they are really concerned about is how capital should be allocated among multiple assets, and how that allocation affects the performance of the entire portfolio. In classical portfolio theory, Markowitz laid the groundwork for this view through mean-variance analysis, showing that returns and risks need to be considered simultaneously and not separately [6]. However, the real market does not operate in a fixed state, so the problem faced by investors is much more complex than the classical form.

Asset prices change, the relationships between assets change, and market conditions rarely remain stable long enough for an initial allocation plan to remain relevant. Therefore, in practice, portfolio allocation should be understood as a dynamic process, where portfolio weighting needs to be adjusted over time as new information emerges.

Simultaneously, the development of machine learning and deep learning has significantly expanded the approach to financial data. However, much of recent research still revolves around forecasting problems, such as forecasting price, return, volatility, or the upward or downward direction of individual assets. These approaches may be useful, but they do not coincide with the problem posed in this thesis. This thesis therefore treats the problem as dynamic portfolio allocation rather than stock

price prediction.

When the problem is stated in that way, reinforcement learning becomes a suitable framework. In reinforcement learning, an agent learns by observing a state, choosing an action, and receiving a reward from the environment [1]. A portfolio weight decision does not only affect the return at the next step; it also changes the portfolio position that will be carried into later rebalancing decisions. For this reason, deep reinforcement learning has been used in studies on portfolio management and dynamic portfolio rebalancing [7, 8].

However, a policy can only be good if the information it receives is sufficiently useful. Market data of many assets often contains time dependencies, cross-relationships between assets, and contextual signals. These factors are not always well captured by manual features or simple models. Transformers are a noteworthy choice in that context, as the self-attention mechanism allows the model to learn relationships in the data series more flexibly [5]. Furthermore, Transformer encoders are often used as a learning module to represent downstream tasks [4]. In this thesis, the Transformer is used in that spirit: not to directly predict prices, but to act as a market representation encoder. At the same time, the use of self-attention also provides an interpretability signal for the proposed framework. By examining attention weights, the study can inspect which parts of the historical input receive greater emphasis during state encoding. Although such analysis is not treated as a complete causal explanation of model behavior, it provides a useful way to relate portfolio decisions to the underlying market information processed by the encoder [4, 5].

Besides the issue of representing state, there is another practical factor that cannot be ignored, which is transaction cost. A strategy may look very attractive if only looking at gross profit, but it will be much less convincing when considering the costs incurred from rebalancing too frequently. Previous studies have shown that transaction cost has a significant impact on portfolio decisions [10, 11]. Therefore, if the learned policy is to have more practical meaning, the model should not optimize profit alone.

For these reasons, this thesis builds a Transformer-based deep reinforcement learning framework for the dynamic portfolio allocation problem. In that frame-

work, the Transformer encoder is used to extract the latent representation of market state from historical multi-asset data, and the PPO-based policy is used to generate portfolio weights. Proximal Policy Optimization (PPO) was chosen because it is a fairly common policy optimization method and usually provides relatively stable update processes [2, 3]. At the same time, transaction cost was included in the reward design so that the policy learned is closer to the actual portfolio allocation conditions.

1.2 Limitations of existing approaches

Although there is now considerable research in quantitative finance, machine learning, and portfolio management, there are still some notable limitations when considering the issue from a dynamic portfolio allocation perspective.

First, many existing studies in financial machine learning are still centered on prediction. The usual target is the price, return, or movement direction of a single asset. These predictions can be useful, but they do not automatically answer the portfolio allocation question. In a portfolio setting, performance also depends on how capital is distributed across assets, how often the allocation is changed, and how much cost is created by rebalancing. This is why recent portfolio optimization studies tend to separate predictive modeling from portfolio policy learning, even though the two tasks are still related [15].

Second, the problem formulation in many studies does not fully reflect the sequential nature of the issue. Portfolio allocation is not a one-time decision. It is an iterative process where current actions influence the results in subsequent steps. Meanwhile, reinforcement learning is properly designed for this type of decision problem, where the goal is to optimize long-term rewards [1]. Therefore, if the portfolio problem is not stated sequentially, it is easy to optimize something that is not its true nature.

Another limitation lies in the market state representation. In the dynamic allocation problem, the model needs to compress information from multiple assets over a retrospective period into a state that is sufficiently useful for decision-making. If

this representation is weak, the underlying policy is unlikely to be strong. Transformers offer a potential direction for learning contextual sequence representations [4, 5], but their role as a state encoder for the portfolio problem is not always clearly emphasized.

Finally, transaction costs are often oversimplified, or even ignored in some studies. This creates a gap between theoretical and practical efficiency. An excessively rebalancing policy can yield good results in backtesting, but when transaction costs are included, that effectiveness can be significantly reduced. Classical and recent studies have shown that this is not a secondary factor [10, 11]. More recent work in DRL-based portfolio selection and dynamic asset allocation continues to support the view that transaction costs materially affect portfolio behavior and should be reflected directly in the problem formulation [17, 18].

1.3 Research problem

This thesis studies the dynamic portfolio allocation problem in a multi-asset context. At each decision-making point, the model observes recent historical data of multiple assets and needs to determine how to allocate capital among them. The goal is not to accurately predict the future price of each individual asset, but to learn a policy that can create portfolio weights that help the portfolio perform well over time.

The problem is stated as sequential decision-making. In reinforcement learning, the agent observes the state, performs an action, and then receives a reward from the environment [1]. In this thesis, the state is constructed from a historical multi-asset data matrix, and the action is a portfolio weight vector. In other words, the model’s output is a capital allocation decision, not an asset-level forecast.

A major challenge lies in how to construct the state. Market data contains time dependencies, cross-relationships between assets, and contextual signals. Therefore, the model needs a better representation than simply looking at individual observations. This is why the thesis uses a Transformer encoder to transform historical data into a latent contextual representation [4, 5]. In the proposed framework, this

representation acts as the market state for the next decision-making step, rather than a direct forecast output.

The second challenge lies in the learning objective. Profit is not meaningful enough if it is achieved solely through excessive trading. An overly aggressive rebalancing policy might look effective in backtesting, but will be less convincing when transaction costs are taken into account [10, 11]. Therefore, this thesis designs rewards to reflect both portfolio performance and transaction costs simultaneously. The policy learning process is then performed using PPO in an actor-critic structure [2, 3].

From there, the central research problem of the thesis is stated as follows: how to design a Transformer-based deep reinforcement learning framework for dynamic portfolio allocation, where historical multi-asset data is encoded into a useful market state, allocation decisions are generated through a policy network, and the learning process is driven by a reward that reflects both portfolio performance and transaction costs.

1.4 Research objectives

The overall goal of this thesis is to develop and evaluate a Transformer-based deep reinforcement learning framework for the dynamic portfolio allocation problem. Instead of viewing asset forecasting as the ultimate goal, the research focuses on directly learning portfolio allocation decisions from historical multi-asset data. This perspective aligns with reinforcement learning as a framework for sequential decision-making [1], while leveraging the ability of Transformer encoders to learn representations of sequential data [4, 5].

More specifically, the thesis has four main objectives. First, it aims to design a framework in which Transformer encoders are used as market representation encoders to extract latent state representations from recent historical data [4, 5]. Second, it aims to integrate that representation into a PPO-based policy capable of generating portfolio weights for the multi-asset portfolio allocation problem [2, 3]. Third, it aims to incorporate transaction costs into reward formulation so

that the learned policy more closely matches real-world portfolio behavior [10, 11]. Fourth, it aims to provide a supporting interpretability-oriented analysis by examining attention-based signals extracted from the Transformer encoder, in order to offer qualitative insight into how historical market information contributes to portfolio allocation decisions [5, 14]. This interpretability component is not treated as a separate main research question, but as a supplementary analytical layer that supports the explainability objective of the thesis.

The study does not aim to build another asset price prediction model, nor does it argue that the Transformer should outperform all portfolio allocation strategies. Its focus is narrower: to examine whether Transformer-based state encoding can provide a more useful market representation for a PPO-based portfolio allocation policy than the selected MLP and LSTM encoder baselines. Within this setting, the contribution also includes the incorporation of transaction costs into the reward design and a supporting attention-based analysis of temporal information used by the encoder.

1.5 Research questions

This thesis is guided by two main research questions:

Research Question 1 (RQ1): Does the Transformer encoder improve DRL-based portfolio optimization relative to LSTM- and MLP-based encoders under a common PPO framework [4, 5]?

Research Question 2 (RQ2): How does incorporating transaction-cost penalty into the reward function influence portfolio performance and policy behavior in DRL-based portfolio optimization [10, 11]?

These two questions are directly aligned with the core components of the proposed framework, namely market representation and reward design. In the empirical stage presented in Chapter 4 and Chapter 5, the Transformer encoder is compared with simpler learned encoders, namely an MLP-based encoder and an LSTM-based encoder, so that the contribution of attention-based representation learning to dynamic portfolio allocation can be assessed more explicitly [4, 5, 14].

In addition to these two main questions, the thesis also includes a supporting interpretability-oriented question:

Supporting Question (SQ): What do the temporal attention patterns of the trained Transformer encoder reveal about how historical information is emphasized under different market conditions [5, 14]?

This supporting question is examined using representative attention heatmaps, grouped attention maps, and a comparison between high-volatility and more stable test windows. The aim is not to create another performance measure or to prove the exact reasoning of the model. Instead, this part adds supporting evidence for the explainability aspect of the thesis.

1.6 Overview of the proposed approach

The framework proposed in this thesis combines representation learning and reinforcement learning in a single pipeline for the dynamic portfolio allocation problem. At each decision-making step, the model receives a rolling window of historical multi-asset market data as input. This input is processed by a Transformer encoder, which is used to construct a latent market representation rather than to generate direct price forecasts [4, 5].

The resulting context vector h_t serves as the internal state representation for the decision-making block. Based on this representation, the actor network produces asset preference scores and converts them into portfolio weights through a softmax layer. The critic network estimates state value and supports policy improvement under an actor–critic framework [1, 2, 3].

Once an allocation is made, the market environment returns the realized portfolio outcome at the next step. The reward is designed to reflect both portfolio growth and the cost of rebalancing, so that the learned policy is encouraged to seek beneficial allocations while avoiding unnecessary turnover [10, 11]. The policy is then optimized using PPO, which is adopted for its practical stability in policy-gradient training [2, 3].

Because the state encoder is attention-based, the framework also supports a

qualitative interpretability layer through attention analysis. In the empirical stage, this analysis is conducted primarily along the temporal dimension, allowing the study to examine which lookback positions receive relatively greater emphasis during state construction [5, 14]. Although this analysis cannot fully explain the model’s behavior, it still gives an additional view of how the encoder uses historical market information before the portfolio weights are produced.

1.7 Scope and constraints

The scope of this thesis is dynamic portfolio allocation. Historical market data are used to describe recent market conditions and guide portfolio-weight decisions. They are not used to build a separate model for forecasting the price of each individual stock.

The thesis also does not aim to introduce a new Transformer architecture or a new reinforcement learning algorithm. Instead, it uses existing methods and connects them in a framework designed for the portfolio allocation problem [1, 4, 5].

Although transaction costs are included in the model, other practical factors such as slippage, liquidity constraints, or institutional constraints have not been fully modeled. Therefore, the results of the thesis should be understood within the scope of the data, the proposed framework, and the assumptions used in the experimental setup [10, 11].

1.8 Thesis structure

This thesis is organized into six chapters. Chapter 1 introduces the research background, problem, objectives, research questions, proposed approach, and scope. Chapter 2 reviews the literature on portfolio optimization, machine learning in finance, Transformers, reinforcement learning, and transaction-cost-aware reward design. Chapter 3 presents the proposed methodology, while Chapter 4 describes the experimental setup. Chapter 5 reports and discusses the empirical results. Chapter 6 concludes the thesis and outlines limitations and future work.

Chapter 2

LITERATURE REVIEW

2.1 Introduction

Dynamic portfolio allocation connects several areas of research, including portfolio theory, machine learning, financial time-series modeling, and reinforcement learning under uncertainty [1, 6]. These areas are related, but they do not all focus on the same question. Many studies in financial machine learning are mainly concerned with forecasting, such as predicting price movement, return, or volatility. This thesis takes a different direction. It focuses on how capital should be allocated across several assets over time, rather than on predicting the movement of one individual asset.

Therefore, this chapter reviews the literature needed to position the thesis as a dynamic portfolio allocation study. The focus is on portfolio-level decision-making, market state representation, reinforcement learning, transaction-cost-aware reward design, and attention-based interpretability [1, 2, 3, 4, 5, 10, 11, 14].

2.2 Portfolio optimization and dynamic allocation

A natural starting point for this thesis is portfolio theory. In classical literature, Markowitz articulated the portfolio selection problem by viewing the portfolio, rather than individual assets, as the central decision-making unit, and formalized the trade-off between return and risk through mean-variance analysis [6]. The value

of this contribution lies not only in the specific model, but in the fact that it establishes a fundamental principle: investment decisions should be evaluated at the portfolio level.

In its classical form, mean–variance portfolio selection can be expressed as a trade-off between expected portfolio return and portfolio risk:

$$\begin{aligned} \max_{\mathbf{w}} \quad & \mathbf{w}^\top \boldsymbol{\mu} - \frac{\lambda}{2} \mathbf{w}^\top \Sigma \mathbf{w} \\ \text{subject to} \quad & \sum_{i=1}^N w_i = 1, \\ & w_i \geq 0, \quad i = 1, \dots, N. \end{aligned} \tag{2.1}$$

where $\mathbf{w}$ is the portfolio weight vector, $\boldsymbol{\mu}$ is the vector of expected asset returns, Σ is the covariance matrix, and λ controls the degree of risk aversion. This formulation makes the portfolio-level return–risk trade-off explicit. However, it is still essentially a static allocation view, whereas the present thesis focuses on repeated allocation decisions over time.

This classical view reveals limitations when placed in the context of the real market. Portfolios are not established once and for all. Asset prices change, the relationships between assets change, and market conditions shift over time. Therefore, capital allocation decisions often have to be adjusted when new information emerges. From this perspective, portfolio allocation is more reasonable to understand as a continuous process rather than a one-time optimization problem [1, 6].

Once the problem is viewed in that way, its nature also changes. The model not only needs to find reasonable portfolio weights at a given time, but also needs to learn how to react to market fluctuations, adjust the portfolio as needed, and take into account the cost of doing so. This is why the problem becomes a sequential decision-making problem, rather than static optimization [1].

2.3 Machine learning and deep learning in finance

The growing use of machine learning and deep learning in finance has expanded the ways in which historical market data can be modeled. Instead of relying en-

tirely on linear assumptions or hand-designed statistical structures, learning-based methods can capture patterns that are more difficult to specify explicitly. Even so, a clear pattern can be observed in the literature: many studies are designed around prediction rather than portfolio-level decision-making.

2.3.1 Prediction-oriented financial modeling

Much of financial machine learning focuses on forecasting future prices, returns, volatility, or market direction. These tasks are usually treated as supervised learning problems and evaluated with prediction-based metrics such as accuracy, Root Mean Squared Error, or directional accuracy [4, 5].

This thesis focuses on a different problem. In multi-asset allocation, a forecast for one asset does not directly determine the weights of the whole portfolio. The main decision is how capital should be distributed across assets as market conditions change. Therefore, forecasting models are reviewed as related background, while the thesis focuses on policy learning for dynamic portfolio allocation [15].

2.3.2 Sequence modeling approaches in finance

Because financial data is sequential in nature, many types of sequence models have been used in the literature. Sequence modeling architectures, including recurrent neural networks, LSTM-based models, gated recurrent unit models, convolutional neural network-based temporal models, and more recently attention-based models, have all been applied. Each group of models has its own strengths: recurrent models are suitable for sequential structures, convolutional models can exploit local patterns over time, and attention-based models are more suitable when important dependencies are not closely spaced in the sequence [4, 5].

What this thesis is concerned with is not just which type of sequence model to use, but what to use it for. In many studies, sequence models are used as predictors. In this thesis, however, sequence models are used to build state representation for allocation decisions. This point changes the role of the model: from a predictor to a representation learner.

2.4 Attention mechanisms and Transformers

2.4.1 Attention fundamentals

Attention mechanisms allow the model to assign different levels of importance to different parts of the input when constructing representations. Earlier attention-based sequence models showed that neural networks can improve representation quality by learning which parts of the input should receive greater emphasis during context construction [19]. In self-attention, each element in the sequence can reference other elements to update its own representation [5]. This creates a clear advantage over purely sequential processing, as information can be directly linked across the entire observation history.

For financial data, this characteristic is significant because useful signals are not always found in the most recent observations. There are patterns that only become meaningful when placed in relation to other points in history. Therefore, attention is not just a technical improvement, but a suitable mechanism for constructing more contextually rich market representations.

At the same time, attention also offers a useful interpretability signal. Since self-attention assigns different weights to different parts of the input sequence, it allows the researcher to inspect which lookback positions receive relatively greater emphasis during representation learning [5, 19, 20]. In the present thesis, such attention-based analysis is interpreted as a qualitative diagnostic signal rather than as a complete causal explanation of model behavior [5, 14].

2.4.2 Transformer encoder architecture

The Transformer architecture proposed by Vaswani et al. is built from multi-head self-attention and position-wise feed-forward networks, combined with residual connections and normalization [5]. This design allows the model to generate contextual representations without relying heavily on recurrent states.

In deep learning literature such as Dive into Deep Learning, the Transformer encoder is also presented as a representation learning module that can be used

for many downstream tasks [4]. The study does not need a complete sequence-to-sequence model, nor does it need a Transformer to directly predict prices. What the thesis needs is a mechanism capable of transforming historical multi-asset data into a latent representation to serve subsequent allocation decisions.

2.4.3 Transformer as a market representation learner

In financial applications, Transformer models are commonly discussed in relation to forecasting tasks, where the main concern is whether the model can predict prices, returns, volatility, or market direction more accurately. This thesis uses the Transformer in a different role. The encoder is not treated as a forecasting engine. Instead, it is used to convert recent multi-asset information into a market representation that can support the portfolio policy. This interpretation is closer to the purpose of the present study, and it is also consistent with recent portfolio optimization research that applies attention-based Transformer architectures to capture temporal and cross-asset structure [14].

Conversely, a representation encoder transforms historical data into a potential state to support decision-making. In the thesis framework, the Transformer encoder processes multi-asset historical data and creates a context vector h_t , which acts as an internal representation of the market state before portfolio allocation [4, 5].

In this understanding, the value of the Transformer no longer lies in how well it forecasts prices. Instead, it is evaluated by whether the representation it creates helps policy generate better portfolio weights. In addition, because the representation is built through self-attention, the encoder also creates an opportunity for interpretability analysis by revealing which parts of the historical input are relatively more influential during state construction [5, 14].

In the present study, the Transformer encoder is ultimately interpreted as a temporal market representation learner, because the empirical attention analysis is conducted primarily over lookback positions rather than as a direct asset-to-asset attribution mechanism [4, 5, 14].

2.5 Reinforcement learning for portfolio management

2.5.1 Reinforcement learning as a sequential decision framework

Reinforcement learning is particularly well-suited to problems where current actions affect future outcomes. In RL, the agent observes the state of the environment, selects an action, and then updates its behavior based on the reward it receives [1]. This is very close to dynamic portfolio allocation, where each allocation decision affects not only the current outcome but also the portfolio structure and subsequent trading behavior.

Instead of learning a signal first and then translating that signal into a portfolio rule, the model can directly learn the allocation policy from the portfolio outcome it generates. Earlier work in financial trading had already suggested that reinforcement learning could be used to optimize sequential trading behavior directly rather than relying only on static prediction rules [21].

2.5.2 Deep Reinforcement Learning in Portfolio Management

When the portfolio universe becomes larger and market conditions are harder to describe with simple rules, a basic reinforcement learning setup may not be sufficient. This is where deep reinforcement learning becomes useful for portfolio management. Instead of depending only on a separate forecasting model, the policy can be trained from the portfolio process itself. It learns from the link between allocation decisions, market movements, and the rewards received after those decisions.

This change is important for the present thesis. In portfolio management, the main question is not only whether one asset is likely to increase or decrease. The more relevant question is how capital should be distributed across several assets as conditions change over time. Earlier studies already followed this direction. Jiang,

Xu, and Liang proposed a deep reinforcement learning framework for portfolio management, where the allocation policy is learned directly in a sequential setting [7]. Lim et al. studied dynamic portfolio rebalancing with reinforcement learning and also focused on repeated allocation decisions rather than isolated forecasts [8]. Later studies continued this line of work. Reinforcement learning has been applied to equity market neutral portfolio management [16], while recent DRL-based portfolio studies have also included transaction cost and risk-related considerations in the learning process [17].

Even so, one issue still remains visible in much of this literature. In many cases, the main attention is given to the policy update itself, while the quality of the state representation is not emphasized to the same degree. But in portfolio allocation, state representation is not a secondary detail. The policy can only act on what the model is able to represent from historical market information. For that reason, the current thesis is positioned a little differently. It does not try to introduce a new reinforcement learning algorithm. Instead, it keeps PPO as the policy optimization method and places more emphasis on strengthening the market representation through a Transformer-based encoder.

2.5.3 PPO for policy optimization

In the group of policy gradient methods, PPO is often considered a practical and relatively stable choice. Schulman et al. introduced PPO as a method to control the degree of policy change through clipped objective [3]. In practical literature such as *Spinning Up in Deep Reinforcement Learning*, PPO is also considered a reliable choice for stable policy optimization [2].

For this thesis, the use of PPO is not intended to create novelty at the algorithmic level. The reason for choosing PPO is that it provides a reasonable optimization mechanism for a weight-based policy with a continuous output. This allows the thesis to focus more on two important components with their contributions: market representation and reward design.

2.6 Transaction costs and practical reward design

In the portfolio problem, transaction cost is not a secondary detail. More recent studies in both DRL-based portfolio selection and dynamic asset allocation continue to show that transaction costs materially affect portfolio behavior and should be treated as part of the problem formulation [17, 18]. If the model only focuses on portfolio return before costs, it may learn a policy that continuously changes portfolio weights in order to exploit short-term fluctuations. In a simulated setting, such behavior can sometimes produce good results. However, once the same policy is considered in a real trading context, the costs generated by repeated rebalancing can materially change those results [10, 11].

For that reason, transaction cost needs to be incorporated directly into the reward design. In the framework proposed in this thesis, reward reflects not only portfolio growth but also penalizes excessive changes between two consecutive portfolio weight vectors. This means that the model is encouraged not only to improve return, but also to avoid unnecessary rebalancing.

Using the difference between w_t and w_{t-1} to measure portfolio change is reasonable in this setting because it directly reflects how much adjustment the policy makes between two decision steps. When the change is large, the penalty becomes larger. When the portfolio remains more stable, the cost deducted from reward is smaller. In that sense, this is a simple way to incorporate transaction costs into the learning process, while still remaining close to the structure of the actual problem [10, 11].

In reinforcement learning, reward does more than evaluate the outcome after an action is taken. It also influences the kind of behavior the policy is likely to learn. If state representation determines what the agent observes from the market, then reward helps determine how the agent responds to that information. From this perspective, incorporating transaction cost into reward is not a small adjustment at the end of the pipeline. It is part of formulating the dynamic portfolio allocation problem in a more practical way.

2.7 Literature Synthesis and Research Gap

The studies reviewed in this chapter show why this thesis needs to combine ideas from portfolio theory, representation learning, and reinforcement learning. Portfolio theory gives the basic investment view: the decision should be evaluated at the portfolio level, not only at the level of separate assets [6]. However, the setting changes once portfolio weights have to be updated again and again as market conditions move. At this stage, portfolio allocation is no longer just a static optimization problem. It becomes a repeated decision problem, where each allocation is made under changing market conditions. This is the reason reinforcement learning is relevant to the thesis, since RL is designed for problems where actions and rewards unfold over time [1].

The reviewed studies also point to a gap in financial machine learning. A large part of the literature still focuses on prediction, for example forecasting the price, return, volatility, or direction of one asset. These outputs may be useful as market signals, but they do not directly decide how capital should be split across several assets. In this thesis, the allocation decision is the main focus. This is also consistent with recent portfolio studies, which discuss forecasting and allocation as related tasks, but not the same task [15].

Another important point is state representation. In a sequential allocation setting, the policy can only make a good decision if the market state is represented well enough. If the encoded state misses useful information from recent market movements, the policy trained on top of it will also be limited. Transformer-based architectures provide a powerful mechanism for contextual sequence representation through self-attention [4, 5], and recent portfolio studies suggest that attention-based encoders can be used meaningfully in this setting [12, 14]. The second issue is transaction cost. It is not a minor practical detail, because a policy that looks attractive before trading frictions may become much less convincing once rebalancing costs are taken into account [10, 11, 17, 18].

For that reason, the gap addressed in this thesis is not the absence of another forecasting model. The gap lies in the need for a controlled portfolio-allocation

framework that connects three elements in one setting: market state representation, reinforcement-learning-based portfolio action, and transaction-cost-aware reward design. Within this framework, the Transformer encoder is evaluated not as a price predictor, but as a market representation module whose value is judged by downstream portfolio performance. The attention mechanism is then used as a limited diagnostic tool for interpreting temporal emphasis in the encoded state.

This is also where the contribution of the thesis becomes clearer. The study is not trying to introduce a new reinforcement learning algorithm, nor does it design a new Transformer architecture. Its focus is more practical: to test whether an attention-based temporal encoder, when combined with PPO, can produce a better learned allocation policy than simpler encoder choices. The evaluation is kept at the portfolio level by considering transaction costs, benchmark strategies, and attention-based interpretation together [5, 14].

2.8 Chapter Summary

This chapter reviewed the main literature supporting the thesis, including portfolio theory, machine learning and deep learning in finance, Transformer-based representation learning, reinforcement learning for portfolio management, and transaction-cost-aware reward design. The review shows that the thesis is not positioned as a stock prediction study, but as a dynamic portfolio allocation problem [1, 6].

Two points are especially important for the proposed framework. First, the quality of the portfolio policy depends on how recent multi-asset market information is represented [4, 5, 12, 14]. Second, transaction cost should be included in the learning objective because rebalancing behavior directly affects practical portfolio performance [10, 11, 17, 18]. These points motivate the Transformer-based state encoder and cost-aware PPO framework developed in Chapter 3.

Chapter 3

PROPOSED METHODOLOGY

3.1 Chapter Overview

This chapter presents the proposed methodology for the dynamic portfolio allocation problem. The framework combines a Transformer-based state encoder with an actor-critic reinforcement learning architecture optimized by PPO. The role of the methodology chapter is to formalize the portfolio decision problem, define the main components of the model, and specify how portfolio performance and transaction cost are incorporated into the learning process [1, 2, 3, 5, 10, 11].

The chapter proceeds from the problem formulation to the model structure. It first defines the allocation task as a sequential decision-making problem. It then specifies the state representation, the portfolio action, the Transformer-based market representation encoder, the actor-critic architecture, the return and transaction cost formulation, the reward function, and the PPO-based policy optimization procedure. Because the thesis also includes an explainability component, the chapter further introduces attention-based interpretability as a supporting analytical layer.

Overall, this chapter focuses on how the proposed framework is constructed and why its main methodological components are appropriate for the research problem addressed in the thesis.

In this chapter, the mathematical formulation and the proposed methodology are presented together because they are closely connected in the portfolio allocation framework. The portfolio problem is first expressed through the long-only, fully in-

vested portfolio-weight simplex and the Markov Decision Process formulation. The Transformer encoder is then introduced as the state representation module, while the actor–critic policy and PPO objective define how portfolio weights are learned. Finally, the transaction-cost-aware reward function links the mathematical formulation to the practical rebalancing behavior evaluated in the empirical chapters.

3.2 Problem Formulation

3.2.1 Dynamic Portfolio Allocation

In this thesis, the problem under consideration is dynamic portfolio allocation over a universe of N tradable assets. At each decision time t , the agent observes recent market information and determines how the available capital should be allocated across the assets for the next trading period. Therefore, the output of the model is not a price forecast or a directional prediction for an individual stock, but a portfolio weight vector that represents the capital allocation decision at that time.

Let the asset universe be denoted by

$$\mathcal{A} = \{1, 2, \dots, N\}. \quad (3.1)$$

The portfolio decision at time t is represented by

$$\mathbf{w}_t = [w_{t,1}, w_{t,2}, \dots, w_{t,N}]^\top, \quad (3.2)$$

where $w_{t,i}$ denotes the proportion of capital allocated to asset i . In this study, the portfolio is constructed under a long-only and fully invested setting. This means that short selling is not considered and the entire capital is allocated across the portfolio at each decision step. As a result, the weight vector must satisfy the following constraints:

$$w_{t,i} \geq 0, \quad \sum_{i=1}^N w_{t,i} = 1. \quad (3.3)$$

Accordingly, the action space of the problem is the set of all valid portfolio weight

vectors on the simplex:

$$\Delta^{N-1} = \left\{ \mathbf{w} \in \mathbb{R}^N \mid w_i \geq 0, \sum_{i=1}^N w_i = 1 \right\}. \quad (3.4)$$

This formulation is consistent with the nature of portfolio selection, where the central objective is to determine how wealth should be distributed across assets under investment constraints, rather than to optimize isolated trading decisions for individual securities [6]. It is also consistent with recent studies on portfolio management using deep reinforcement learning, where the policy is learned directly to produce portfolio allocation decisions over time [7, 8, 12, 13].

3.2.2 Markov Decision Process Formulation

The portfolio problem considered in this thesis is not a one-step choice. Once an allocation is made, its effect does not stop at the current trading period. A portfolio action does not end at the moment it is chosen. It changes the current portfolio value, becomes the starting point for the next rebalancing decision, and may also create future trading costs. For this reason, the problem is modeled as a Markov Decision Process (MDP), which is a suitable framework for sequential decision-making under uncertainty [1].

The environment is represented as

$$\mathcal{M} = (S, W, P, R, \gamma), \quad (3.5)$$

where S is the state space, W is the set of valid portfolio-weight actions, P describes the transition dynamics, R is the reward function, and $\gamma \in (0, 1]$ is the discount factor.

At each time step t , the agent observes the current state s_t , selects a portfolio weight vector w_t , receives the reward R_t , and then observes the next state s_{t+1} . The policy π_θ is trained to maximize the expected discounted return:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t R_t \right]. \quad (3.6)$$

This formulation fits the thesis because a portfolio decision should not be evaluated only by its immediate return. A weight vector that looks good in one period may still be undesirable if it leads to unstable rebalancing or repeated transaction costs later. Therefore, dynamic portfolio allocation is treated here as a long-horizon control problem, rather than as a set of separate one-step decisions [1, 9, 13].

3.3 State Representation

At each time step t , the agent does not observe only one market snapshot. Instead, it receives a rolling historical window that contains the most recent L periods for N tradable assets. This input is written as

$$X_{t-L:t}. \tag{3.7}$$

In the implementation, $X_{t-L:t}$ is stored as a tensor in $\mathbb{R}^{L \times N \times F}$. Here, L denotes the lookback length, N denotes the number of assets, and F denotes the number of features for each asset. The feature set can include information derived from market data, such as normalized prices, simple returns, log returns, volume-related variables, momentum indicators, moving averages, and volatility-related measures. Chapter 4 explains the exact feature construction in detail. At the methodology level, the role of this input is to give the policy a structured description of recent market conditions across multiple assets.

Accordingly, the state at time t is defined as

$$s_t \equiv X_{t-L:t}. \tag{3.8}$$

This state definition is appropriate because dynamic portfolio allocation depends on recent market context rather than on a single current observation. In a multi-asset setting, the portfolio decision is shaped not only by the temporal behavior of each asset individually, but also by the relative behavior among assets within the investment universe [7, 8, 12, 14].

Although the historical input contains the relevant market information, it is not

passed directly to the portfolio policy. Instead, it is first transformed into a more informative latent representation through the Transformer encoder. This intermediate step is important because financial market data are noisy, high-dimensional, and heterogeneous, so an explicit representation learning stage can improve the quality of the information used for downstream decision making [1, 4].

For notational simplicity in the experimental sections, the rolling state tensor may also be written directly as $X_t \in \mathbb{R}^{L \times N \times F}$, with the understanding that it represents the historical window $X_{t-L:t}$ used at decision time t .

3.4 Assumptions, Constraints, and Notation

To keep the portfolio problem within a clear and manageable scope, this study uses several simplifying assumptions. The portfolio is long-only, so the model is not allowed to take short positions. It is also fully invested at every decision step, meaning that all available capital is distributed across the selected assets. Rebalancing is assumed to occur at discrete time steps, which matches the sequential setting used in the reinforcement learning formulation. Transaction cost is included through a proportional penalty based on how much the portfolio weights change from one decision step to the next. These assumptions make the experiment more practical than a frictionless setup, while still keeping the model design and empirical evaluation tractable [6, 10, 11, 13].

The main notation used throughout the thesis is as follows:

- $X_{t-L:t}$: historical market data from time $t - L$ to time t
- h_t : context vector produced by the Transformer encoder
- z_t : unconstrained asset score vector generated by the actor
- w_t : portfolio allocation at time t
- y_t : vector of realized asset gross returns at time t
- r_t : vector of realized asset simple returns at time t

- G_t : portfolio growth generated by w_t and y_t
- R_t : reward at time t
- c : transaction cost coefficient
- π_θ : policy parameterized by θ
- V_ϕ : critic or value function parameterized by ϕ
- $\hat{A}_t$: estimated advantage at time t
- $\hat{G}_t$: empirical return target at time t
- s_t : state observed at decision time t
- $\rho_t(\theta)$: PPO probability ratio

The same notation is used in the later chapters to avoid ambiguity between the methodology, implementation, and result discussion.

3.5 Transformer-Based Market Representation Encoder

In a multi-asset portfolio, the input data are not only time series for separate assets. They also contain movements that may be related across assets. For this reason, the encoder should be able to read the historical window over time while still preserving information about the asset universe. This is the main reason the thesis uses a Transformer encoder in the proposed framework.

In the proposed framework, the historical input $X_{t-L:t}$ is first transformed into a sequence of embedding vectors before being processed by the Transformer encoder. Each embedding vector represents market information extracted from the original observation tensor after projection into the model space. Through this step, the raw multi-asset historical input is converted into a token sequence that can be handled by self-attention while preserving the information needed for sequential portfolio decisions.

In the implementation used in this thesis, the historical input at each time step is first represented as a flattened asset–feature slice, and the Transformer encoder then processes the resulting sequence across lookback positions. Accordingly, the effective token sequence is organized primarily along the temporal dimension. This design allows the encoder to model temporal dependencies across historical portfolio-relevant market states while still preserving multi-asset information inside each time-step representation [4, 5, 12, 14].

Let the embedded input sequence be denoted by

$$X \in \mathbb{R}^{n \times d_{\text{model}}}, \quad (3.9)$$

where n is the effective sequence length after tokenization and d_{model} is the hidden dimension. Positional information is incorporated into the input sequence so that the encoder can distinguish the relative ordering of observations within the lookback window.

Before entering the Transformer encoder, each tokenized input vector x_i is projected into the model space through a learnable linear transformation,

$$e_i = W_e x_i + b_e, \quad (3.10)$$

where W_e and b_e denote the embedding parameters. The resulting embedded sequence is then used as input to the self-attention layers.

The Transformer is built around the self-attention mechanism, which allows each element in an input sequence to attend to all other elements in the same sequence [5]. In self-attention, the embedded input is linearly projected into query, key, and value matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (3.11)$$

where W^Q , W^K , and W^V are learnable parameter matrices.

The scaled dot-product attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (3.12)$$

This mechanism enables each position in the sequence to aggregate information from all other positions according to learned relevance weights [5]. Compared with strictly sequential architectures, self-attention provides a more flexible way to capture long-range temporal dependency and contextual interaction across the input sequence [4, 5].

Recent portfolio management studies also support the use of attention-based representation learning in sequential allocation problems. Li et al. employed a Transformer-based long-sequence representation extractor together with cross-asset attention for online portfolio management with high-frequency data [12]. Kim and Khushi also proposed an attention-based Transformer architecture for portfolio optimization under realistic trading frictions [14]. These studies strengthen the methodological rationale for using a Transformer encoder as the representation module in the present work.

In this framework, the Transformer is not trained to predict the next price or return directly. Its role is earlier in the decision pipeline: it takes the historical input $X_{t-L:t}$ and turns it into a latent market state for the policy network. This distinction matters because the thesis is concerned with learning portfolio allocations. The encoder is useful only if the representation it produces helps the downstream policy make better allocation decisions, rather than because it solves a separate forecasting problem [7, 8, 9, 12, 14].

The output of the Transformer encoder is denoted by

$$h_t = f_\psi(X_{t-L:t}), \quad (3.13)$$

where $f_\psi(\cdot)$ is the encoder parameterized by ψ . The vector h_t serves as the latent market representation used by both the actor and the critic networks.

The encoder produces a contextual latent state that is more suitable for downstream portfolio decision-making. This design is consistent with the objective of the

thesis, where the main goal is to construct an informative state representation for sequential allocation decisions rather than to model asset-level forecasting directly. In implementation terms, the construction of the Transformer encoder can also be informed by publicly available annotated reference code that closely follows the original architecture and helps clarify the interaction between attention, positional encoding, and feed-forward layers [5, 25].

3.6 Attention-Based Interpretability

Since the thesis is positioned as an explainable deep reinforcement learning framework, interpretability is incorporated through the analysis of attention patterns produced by the Transformer encoder. In the proposed model, attention weights are extracted from the self-attention layers and used as an auxiliary signal for examining how the context vector h_t is formed from the historical input $X_{t-L:t}$ [5, 14].

In the implementation adopted in this thesis, the Transformer encoder operates primarily over the temporal dimension after each time step is represented as a flattened asset-feature slice. Therefore, the extracted attention maps should be interpreted mainly as *temporal attention maps across lookback positions*, rather than as direct asset-to-asset attribution maps. Under this interpretation, attention analysis provides a qualitative view of how the encoder distributes emphasis across historical periods before the portfolio allocation w_t is generated [4, 5, 14].

The interpretability analysis is carried out in two ways. The first is to inspect attention heatmaps from selected out-of-sample windows. These examples give a direct visual view of how attention is distributed in individual test cases. The second is to average attention maps across groups of windows, including random, high-volatility, and low-volatility groups. This grouped view is useful because it avoids relying too much on a few selected examples and gives a more stable picture of how temporal attention changes across market conditions [5, 14].

The attention weights should not be read as a complete explanation of the policy decision. They cannot tell exactly why the actor assigns a particular portfolio weight at a given step. In this thesis, they are used in a narrower sense: to inspect which

positions in the historical window receive more emphasis when the Transformer builds the state representation [5, 14]. Therefore, the attention analysis is treated as supporting interpretability evidence, not as another performance measure.

3.7 Actor–Critic Architecture and Portfolio Action

The model follows an actor–critic structure. The actor is responsible for producing portfolio weights, while the critic gives an estimate of the value of the current state [1]. This setup is useful here because portfolio rewards are noisy and one allocation may affect later rebalancing decisions. The critic therefore gives an additional reference signal during policy learning, instead of letting the actor depend only on the immediate reward.

There are three main components in the architecture: the Transformer encoder f_ψ , the actor network g_θ , and the critic network V_ϕ . The encoder first converts the historical market window into the context vector h_t . This same vector is then passed to both decision blocks. The actor uses it to generate the portfolio allocation, while the critic uses it to estimate the state value. In this way, representation learning, action generation, and value estimation are separated in function, but they still share the same encoded market state [1, 2, 3].

The actor receives the context vector h_t and produces unconstrained asset preference scores:

$$\mathbf{z}_t = g_\theta(h_t), \quad \mathbf{z}_t \in \mathbb{R}^N. \quad (3.14)$$

These scores are not portfolio weights yet, since the portfolio is restricted to be long-only and fully invested. They are therefore normalized through a softmax function:

$$w_{t,i} = \frac{\exp(z_{t,i})}{\sum_{j=1}^N \exp(z_{t,j})}, \quad i = 1, \dots, N. \quad (3.15)$$

The final portfolio allocation is therefore

$$\mathbf{w}_t = \text{softmax}(g_\theta(h_t)). \quad (3.16)$$

With this mapping, every weight is nonnegative and the weights sum to one. This matches the long-only, fully invested setting used in the thesis. The softmax transformation also keeps the meaning of the actor scores: assets with larger learned scores receive larger portfolio weights [4, 5].

The critic network estimates the value of the current state:

$$V_\phi(s_t) = V_\phi(h_t), \quad (3.17)$$

where ϕ denotes the critic parameters.

The critic does not choose portfolio actions directly. Instead, it estimates expected future return under the current policy. This value estimate serves as a baseline for policy improvement and reduces variance in gradient estimation during training [1, 2]. In this way, the actor determines how capital is allocated, while the critic evaluates the quality of the current state under the learned policy. This actor-critic design is also consistent with recent portfolio reinforcement learning formulations using PPO under transaction costs [13].

3.8 Portfolio Return and Transaction Cost Modeling

Let

$$\mathbf{y}_t = [y_{t,1}, y_{t,2}, \dots, y_{t,N}]^\top \quad (3.18)$$

denote the vector of asset gross returns realized at time t , where

$$y_{t,i} = \frac{p_{t,i}}{p_{t-1,i}}. \quad (3.19)$$

Given a portfolio allocation $\mathbf{w}_t$, the one-period portfolio growth is defined as

$$G_t = \mathbf{w}_t^\top \mathbf{y}_t. \quad (3.20)$$

The quantity G_t represents the gross portfolio growth generated by the allocation

vector and the realized asset returns over the trading period. It is consistent with the standard portfolio view in which performance is jointly determined by allocation weights and asset returns [6, 7].

Let V_t denote the portfolio wealth at time t . Ignoring transaction cost temporarily, wealth evolves according to

$$V_t = V_{t-1}G_t. \quad (3.21)$$

This wealth update is important because portfolio performance is built through compounding. A policy should therefore not be judged only by what happens in one trading step. What matters is how its repeated decisions affect portfolio wealth over the whole evaluation period [6, 8]. This is also why portfolio studies usually consider risk-adjusted performance, not only raw return [23].

Transaction cost is also part of the portfolio problem. If a policy is trained only to increase gross return, it may learn to rebalance too often. Such a strategy can look good in a backtest before costs, but its performance may weaken once trading frictions are included [10, 11]. In reinforcement learning, this issue is especially relevant because the reward function guides the behavior learned by the agent. For that reason, transaction cost is included directly in the portfolio formulation, instead of being added only as a reporting item after training [8, 9, 10, 11, 13, 14].

In this study, transaction cost is modeled through the change in portfolio weights between two consecutive allocation decisions. Let the transaction cost penalty be defined as

$$c\|\mathbf{w}_t - \mathbf{w}_{t-1}\|_1. \quad (3.22)$$

The corresponding one-step portfolio turnover is defined as

$$\tau_t = \|w_t - w_{t-1}\|_1. \quad (3.23)$$

In this thesis, turnover is used both in the transaction-cost penalty and as an evaluation metric for measuring how aggressively the learned policy rebalances the portfolio over time.

The L_1 norm captures the overall magnitude of portfolio reallocation across assets, so larger allocation changes correspond to larger trading costs. This choice is appropriate because the L_1 norm reflects total portfolio turnover in a simple and interpretable form, making it suitable as a proportional approximation of rebalancing cost in a long-only setting [10, 11, 13, 14].

Recent portfolio studies also point in the same direction. Betancourt and Chen include transaction costs in a DRL portfolio setting where the number of assets can change over time [13]. Kim and Khushi make a similar practical point from another angle, noting that portfolio studies often simplify market frictions and should be evaluated under costs such as transaction fees and slippage [14].

3.9 Reward Function

In this methodology, y_t denotes the vector of realized gross returns. In the actual implementation, the environment can also express returns in simple-return form as r_t . When simple returns are used, portfolio growth is written as $1 + w_t^\top r_t$. Since the portfolio is fully invested, this is equivalent to using gross returns, because $w_t^\top y_t = 1 + w_t^\top r_t$. This note is included to make the notation in the methodology match the implementation while keeping the same portfolio meaning [6, 10, 11].

The reward is defined from the portfolio allocation objective. The model is not rewarded for predicting prices or classifying market direction. Instead, it receives feedback from the realized portfolio outcome. At the same time, the reward must take rebalancing cost into account, since a policy that changes weights too often may look good before costs but become less useful once trading frictions are included.

Therefore, the reward has two components. One component measures portfolio growth, and the other penalizes turnover from changes in portfolio weights. At time t , the reward is defined as

$$R_t = \log(G_t) - c\|\mathbf{w}_t - \mathbf{w}_{t-1}\|_1. \quad (3.24)$$

Substituting the definition of G_t gives

$$R_t = \log(\mathbf{w}_t^\top \mathbf{y}_t) - c \|\mathbf{w}_t - \mathbf{w}_{t-1}\|_1. \quad (3.25)$$

This formulation fits the portfolio setting for several reasons. The logarithmic growth term is suitable because portfolio wealth compounds over time, so the reward should reflect growth in a way that is consistent with long-horizon performance. The transaction-cost term then reduces the reward when the policy makes large changes in allocation. This is important because a strategy may appear attractive in a backtest before costs, but become less convincing after repeated trading is penalized [23, 24]. In this sense, the reward is not only an evaluation of the immediate portfolio outcome. It also guides the type of rebalancing behavior that the agent learns during training [8, 9, 10, 11].

The interpretation is direct. If the policy changes the portfolio weights substantially from one step to the next, the penalty increases and the reward becomes smaller. If the policy produces favorable portfolio growth while keeping allocation changes more stable, the reward becomes more favorable. Therefore, the reward remains profit-oriented, but it is also cost-aware, which makes it more appropriate for dynamic portfolio allocation under trading frictions [8, 9, 10, 11, 13, 14].

It should be noted that including a transaction-cost penalty in the reward exposes the policy to rebalancing frictions during training, but it does not mechanically guarantee lower realized turnover in every out-of-sample period. The final trading behavior also depends on the learned market representation, the selected checkpoint, and the realized market path during evaluation.

For consistency, R_t is used in the later chapters whenever the thesis refers to the reward.

3.10 PPO-Based Policy Optimization

To optimize the portfolio policy, this thesis adopts Proximal Policy Optimization (PPO). PPO is a policy-gradient method in the actor-critic family and is widely

used because it offers a practical balance between implementation simplicity and training stability [2, 3]. This choice is suitable for the present study because portfolio allocation is represented by continuous weight vectors, and PPO provides a stable policy optimization mechanism for such continuous decision settings [2, 3, 13]. It is especially suitable for noisy environments, where large and unstable policy updates may seriously damage learning performance.

In the proposed framework, PPO is used to update both the actor and the critic after collecting trajectories of states, actions, and rewards from the market environment. Its defining feature is the clipped surrogate objective, which constrains the effective step size of policy updates and helps keep the new policy close to the old policy [2, 3].

Let $\pi_{\theta_{\text{old}}}$ denote the old policy and π_{θ} denote the updated policy. PPO defines the probability ratio

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (3.26)$$

This ratio measures how much the updated policy changes the probability of the sampled action relative to the policy that generated the data [2, 3].

Before introducing the clipped surrogate objective, it is useful to clarify the role of advantage-based policy updates in PPO. As a policy-gradient method, PPO improves the policy by increasing the probability of actions that lead to higher-than-expected return and decreasing the probability of actions associated with weaker outcomes [1, 2, 3]. In practice, this update relies on an estimate of the advantage function, which measures how favorable a sampled action is relative to the critic's baseline value estimate. This advantage-based formulation is important because it reduces variance in policy-gradient updates and makes actor-critic training more stable in noisy sequential environments such as financial markets [1, 2, 3].

The PPO clipped surrogate objective is given by

$$\mathcal{L}^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (3.27)$$

where $\hat{A}_t$ is an estimate of the advantage function and ϵ is the clipping threshold.

This objective limits the extent to which the updated policy can benefit from

deviating too far from the old policy. As a result, policy optimization becomes more stable and robust, which is particularly useful in financial applications where the reward signal is noisy and nonstationary [2, 3, 9].

The critic estimates the state value $V_\phi(s_t)$, and the advantage can be expressed in simplified form as

$$\hat{A}_t = \hat{G}_t - V_\phi(s_t), \quad (3.28)$$

where $\hat{G}_t$ denotes the empirical return target.

In implementation, the advantage estimate may be obtained through generalized advantage estimation (GAE), which provides a practical bias–variance trade-off for actor–critic learning [3]. This is especially useful in financial reinforcement learning settings, where reward signals are noisy and one-step outcomes may not fully reflect the long-horizon quality of a portfolio decision.

More specifically, the temporal-difference residual is defined as

$$\delta_t = R_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \quad (3.29)$$

The GAE advantage estimate is then computed as

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}. \quad (3.30)$$

Here, λ controls the bias–variance trade-off in the advantage estimate. When λ is larger, the estimate incorporates longer-horizon return information; when it is smaller, the estimate relies more strongly on shorter-horizon temporal-difference information.

The critic is trained by minimizing the value regression loss

$$\mathcal{L}^V(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - \hat{G}_t \right)^2 \right]. \quad (3.31)$$

This actor–critic structure follows the standard reinforcement learning principle in which the critic provides value-based feedback to improve the actor’s policy updates [1, 2]. In addition, the use of PPO in portfolio management has been sup-

ported by recent work that adapts or applies PPO in transaction-aware allocation settings [13].

In practice, the overall PPO training objective can be written as

$$\mathcal{L}(\theta, \phi) = -\mathcal{L}^{\text{clip}}(\theta) + c_v \mathcal{L}^V(\phi) - c_e \mathcal{L}^{\text{ent}}(\theta), \quad (3.32)$$

where c_v is the value-loss coefficient, c_e is the entropy coefficient, and $\mathcal{L}^{\text{ent}}(\theta)$ is an entropy bonus.

The entropy term is included to discourage overly premature policy concentration and to preserve a degree of exploration during training [2, 3]. In financial environments, this component is useful because portfolio decisions are learned under uncertainty, and early collapse to a narrow action pattern may reduce the policy’s ability to adapt to changing market conditions.

3.11 Main Framework of the Proposed Methodology

The complete workflow of the proposed methodology is illustrated in Figure 3.1. At each time step, the model receives historical market data $X_{t-L:t}$ as input. This input is processed by the Transformer encoder to produce the context vector h_t , which serves as the latent market representation for downstream decision-making. Based on h_t , the actor network generates portfolio weights $\mathbf{w}_t$, while the critic estimates state value to support policy improvement [1, 2, 3, 5].

After an allocation is made, the market environment produces the realized portfolio outcome. Portfolio growth is computed from the interaction between $\mathbf{w}_t$ and the realized asset return vector $\mathbf{y}_t$, while transaction cost is approximated from the change between consecutive portfolio weights. These two components are then combined in the reward R_t , which is used to update the actor and critic through PPO [8, 10, 11, 13].

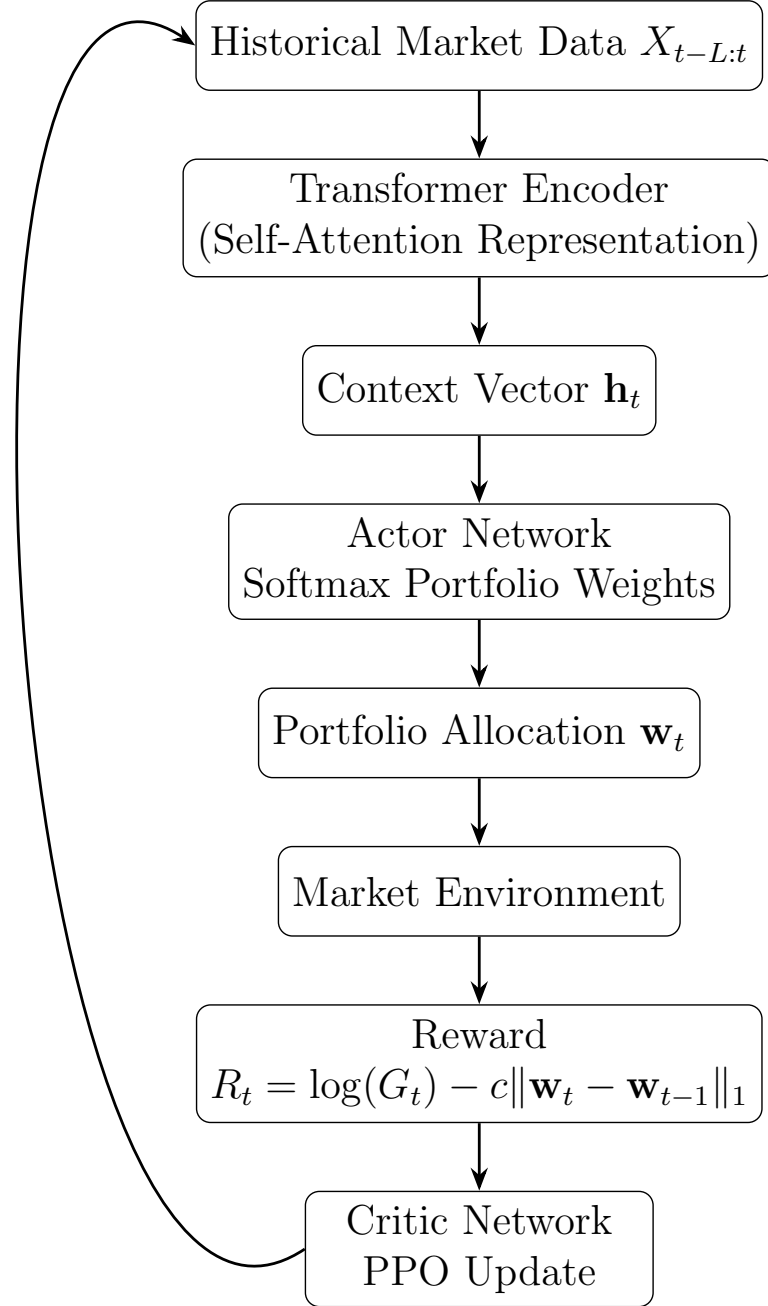


Figure 3.1: Transformer-PPO Portfolio Rebalancing Framework

This workflow can be summarized as

$$X_{t-L:t} \rightarrow h_t \rightarrow \mathbf{w}_t \rightarrow G_t \rightarrow R_t \rightarrow \text{PPO update.} \quad (3.33)$$

The overall workflow of the proposed Transformer-PPO framework is summarized in Algorithm 1.

Algorithm 1 Transformer-PPO Procedure for Dynamic Portfolio Allocation

- 1: **Input:** Historical dataset $\mathcal{D}$, lookback length L , transaction cost coefficient c
 - 2: **Output:** Trained policy π_θ and test-set evaluation results
 - 3: Construct rolling states X_t from historical market data
 - 4: Split $\mathcal{D}$ chronologically into training, validation, and test subsets
 - 5: Initialize the Transformer encoder f_ψ , the actor g_θ , and the critic V_ϕ
 - 6: **for** each training epoch **do**
 - 7: **for** each decision step t in the training environment **do**
 - 8: Encode the current state: $h_t \leftarrow f_\psi(X_t)$
 - 9: Generate portfolio weights: $w_t \leftarrow \text{softmax}(g_\theta(h_t))$
 - 10: Compute portfolio growth: $G_t \leftarrow w_t^\top y_t$
 - 11: Compute reward: $R_t \leftarrow \log(G_t) - c\|w_t - w_{t-1}\|_1$
 - 12: Store trajectories for PPO-based policy improvement
 - 13: **end for**
 - 14: Estimate return targets and advantages from the collected trajectories
 - 15: Update the actor-critic parameters using PPO
 - 16: **end for**
 - 17: Select the best checkpoint on the validation set
 - 18: Evaluate the selected policy on the test set
-

Algorithm 1 shows that the proposed framework integrates state encoding, portfolio action generation, reward computation, and PPO-based policy improvement within a unified sequential allocation loop.

From a methodological perspective, the framework integrates representation learning, portfolio decision-making, and cost-aware reinforcement learning in a single sequential allocation setting. In addition, because the state encoder is attention-

based, the same pipeline also provides an interpretability signal that can later be examined through attention analysis.

3.12 Suitability to Research Questions

The proposed methodology is directly aligned with the two main research questions of the thesis. First, the use of the Transformer encoder to transform historical data $X_{t-L:t}$ into the context vector h_t provides a basis for evaluating whether attention-based market representation improves the effectiveness of the deep reinforcement learning framework for dynamic portfolio allocation when compared with simpler baseline encoders. Second, the reward function R_t incorporates transaction cost through a penalty term on the change in the allocation vector $\mathbf{w}_t$, thereby allowing the effect of transaction-cost-aware reward design on portfolio performance and policy behavior to be examined.

In addition to these two main questions, the methodology also supports a supplementary interpretability-oriented analysis through attention weights extracted from the Transformer encoder. Although this interpretability component is not formulated as a separate main research question, it contributes to the explainability objective of the thesis by providing qualitative insight into how historical market information is emphasized during temporal state construction [5, 14].

3.13 Chapter Summary

This chapter formulated the dynamic portfolio allocation problem as a Markov Decision Process, where the state is a rolling historical market window and the action is a long-only portfolio weight vector. The proposed framework uses a Transformer encoder to construct the latent market state, an actor network to generate portfolio weights, a critic network to estimate state value, and PPO to update the policy [1, 4, 5, 6, 12, 14].

The chapter also defined portfolio growth, transaction cost, and the cost-aware reward function used for training. Attention-based interpretability was introduced

as a supporting diagnostic layer for examining temporal emphasis in the Transformer encoder [2, 3, 10, 11, 13].

Chapter 4

EXPERIMENTAL DESIGN AND IMPLEMENTATION

4.1 Experimental Objectives

This chapter describes the empirical design used to evaluate the proposed Transformer-based deep reinforcement learning framework for dynamic portfolio allocation. The experiments are constructed to address the two research questions stated in Chapter 1. First, the study examines whether a Transformer-based market representation encoder can deliver stronger portfolio performance than simpler learned encoders, specifically MLP- and LSTM-based baselines, when all models are trained under the same PPO framework [3, 4, 5]. Second, the study investigates how the inclusion of transaction cost penalties in the reward function influences out-of-sample portfolio performance and policy behavior [10, 11, 18].

To keep the empirical setting controlled and interpretable, the thesis adopts a fixed five-asset equity universe and evaluates all main experiments on a five-year sample from January 1, 2020 to December 31, 2024. Within this setting, benchmark strategies and learned models are compared under a unified training and evaluation protocol. In addition, the trained Transformer model is examined through an attention-based temporal analysis in order to provide supporting explainability evidence [5, 14, 19, 20]. The overall aim of the experimental design is therefore not only to compare model performance, but also to assess whether the proposed

framework remains coherent from the perspectives of portfolio construction, reward specification, and interpretability.

4.2 Main Dataset Description

The main experiment uses five large U.S. equities: Apple Inc. (AAPL), Microsoft Corporation (MSFT), Amazon.com, Inc. (AMZN), JPMorgan Chase & Co. (JPM), and Johnson & Johnson (JNJ).

Although the universe contains only five stocks, the selected companies still provide useful variation for a controlled allocation experiment. AAPL and MSFT represent large-cap technology firms, AMZN reflects a technology and consumer-platform profile, JPM represents financial services, and JNJ adds a healthcare and defensive-equity component.

Table 4.1: Main five-asset universe used in the empirical study

Ticker	Company	Sector / Profile
AAPL	Apple Inc.	Large-cap technology
MSFT	Microsoft Corporation	Large-cap technology
AMZN	Amazon.com, Inc.	Technology / consumer platform
JPM	JPMorgan Chase & Co.	Financial services
JNJ	Johnson & Johnson	Healthcare / defensive equity

The five selected stocks are chosen for practical reasons. They are large and liquid U.S. equities, with long daily price histories and data that can be accessed from public sources. At the same time, this small universe is not meant to represent the whole equity market.

Daily market data are collected from Yahoo Finance using the `yfinance` interface [26, 27]. The main sample runs from January 1, 2020 to December 31, 2024. Daily frequency is used because the thesis studies portfolio rebalancing over time, not intraday trading.

Before training, the price data are first matched by trading date across the five stocks. The data are then kept in time order, so each rolling state is built only from

information that would have been available at that point. This step is important to avoid look-ahead bias. After that, the sample is divided chronologically into training, validation, and test sets. With the selected lookback window, the final setup gives 717 training steps, 250 validation steps, and 251 test steps.

4.3 Data Preparation and State Construction

Before training, the daily market data are reorganized into rolling windows. At each decision time t , the model receives the most recent L observations for all selected assets and features. The input state is therefore written as

$$X_t \in \mathbb{R}^{L \times N \times F}, \quad (4.1)$$

where L denotes the lookback length, N denotes the number of assets, and F denotes the number of features for each asset.

In the main experiment, the lookback length is set to $L = 20$. This value is used as a fixed design choice for constructing the state, not as a lookback horizon that has been optimized empirically.

The state-construction pipeline is applied consistently across all benchmark and learned-model experiments.

Each sample therefore provides a rolling multi-asset historical window used as the policy input, while the realized next-step asset returns are used for portfolio evaluation and reward computation.

The feature construction is designed to retain recent information relevant to dynamic allocation rather than to maximize the number of technical inputs. In practice, the feature set is derived from daily market observations and transformed into portfolio-relevant signals such as return-based measures, normalized price-related quantities, volatility-related quantities, and trading-activity-related quantities. This design allows the state to capture short-horizon market structure while remaining sufficiently compact for controlled model comparison.

The same underlying state representation is used for all learned encoders. The

MLP baseline processes the input through a feedforward mapping, the LSTM baseline models the state sequentially through recurrent dynamics, and the Transformer encoder processes the input primarily across lookback positions after each time step is represented as a flattened asset–feature slice [4, 5, 14, 25]. Because all models share the same data source, feature construction logic, and rolling-window protocol, the learned-model comparison focuses on the effect of the encoder architecture rather than differences in preprocessing.

The state input is constructed with a 20-day rolling window. At each decision point, the model only receives information from the most recent 20 trading days. Older observations are left outside the current state. This makes the input size fixed across all time steps, which is important because MLP-PPO, LSTM-PPO, and Transformer-PPO need to be compared under the same state-construction setting.

4.4 Portfolio Constraints and Trading Assumptions

The portfolio environment is kept long-only and fully invested. Let $w_t \in \mathbb{R}^N$ be the vector of portfolio weights at time t . A valid action must satisfy

$$w_{t,i} \geq 0, \quad i = 1, \dots, N, \quad (4.2)$$

and

$$\sum_{i=1}^N w_{t,i} = 1. \quad (4.3)$$

These two conditions mean that the model cannot take short positions or use leverage. All available capital is distributed across the selected assets at each decision step.

This setting is used to keep the experiment clear and comparable across strategies. In the implementation, the model output is converted into valid portfolio weights, so the benchmark strategies and learned policies all operate under the same investment constraint. Because of this, differences in the results can be dis-

cussed in terms of the encoder, reward design, and rebalancing behavior, rather than differences in the allowed action space.

The environment is then run as a sequential rebalancing process. At each step, the agent observes the current rolling state, chooses a new portfolio weight vector, and receives the realized portfolio outcome in the following period. This matches the way the thesis formulates portfolio allocation: not as a single static optimization problem, but as a repeated decision-making problem over time [1, 7, 8, 13].

4.5 Benchmark and Model Configurations

4.5.1 Benchmark Strategies

The empirical design includes both benchmark strategies and learned deep reinforcement learning models. The benchmark strategies provide practical and interpretable reference points for assessing the quality of the learned policies.

Three benchmark strategies are considered in the main experiment:

- **Equal Weight:** the portfolio is rebalanced to equal weights across all assets at each decision step;
- **Buy-and-Hold Equal Weight:** the portfolio is initialized at equal weights and then held passively without subsequent rebalancing;
- **Random Policy:** portfolio weights are generated randomly subject to the long-only, fully invested constraint.

The learned models are all trained under the same PPO-based actor–critic framework so that encoder differences can be examined more directly [3, 10, 11]. Three learned encoder configurations are compared:

- **MLP-PPO:** an MLP-based encoder used as a simple feedforward baseline;
- **LSTM-PPO:** an LSTM-based encoder used as a recurrent sequential baseline;
- **Transformer-PPO:** the proposed attention-based encoder used as the main model of the thesis.

This design directly supports RQ1 by examining whether a Transformer-based market representation encoder yields stronger portfolio performance than simpler learned encoders under the same reinforcement learning framework [4, 5, 7, 8, 14]. By fixing the policy optimization framework and varying only the encoder, the empirical comparison remains aligned with the main methodological contribution of the thesis.

These benchmark strategies are included to provide practical reference points for the learned policies. In particular, they help position the Transformer-based policy not only against simpler learned encoders, but also against transparent allocation rules that remain highly relevant in portfolio evaluation.

4.5.2 Learned Encoder Variants

To evaluate the contribution of the proposed Transformer-based state encoder more clearly, the empirical study includes two simpler learned baselines under the same PPO-based reinforcement learning framework, namely *MLP-PPO* and *LSTM-PPO*. These baselines are not introduced as separate forecasting models, but as alternative state encoders within the same dynamic portfolio allocation pipeline. This design is important because the main comparison of the thesis aims to isolate the effect of state representation quality while keeping the downstream policy optimization framework fixed [4, 5].

MLP-PPO baseline.

For the *MLP-PPO* baseline, the rolling state tensor $X_t \in \mathbb{R}^{L \times N \times F}$ is first flattened into a vector and then processed by a feedforward multilayer perceptron. The resulting latent representation is written as

$$\mathbf{h}_t^{\text{MLP}} = f_{\text{MLP}}(\text{vec}(X_t)), \quad (4.4)$$

where $\text{vec}(\cdot)$ denotes the vectorization of the rolling state tensor and $f_{\text{MLP}}(\cdot)$ denotes the nonlinear mapping learned by the MLP encoder. In this formulation, the MLP acts as a simple baseline that compresses the full historical input window

into a latent state representation without explicitly modeling recurrent temporal dynamics or attention-based interactions across lookback positions [4].

More specifically, if the flattened input is denoted by $\mathbf{u}_t = \text{vec}(X_t)$, then a simple multilayer formulation may be expressed as

$$\mathbf{z}_t^{(1)} = \phi(W^{(1)}\mathbf{u}_t + \mathbf{b}^{(1)}) \quad (4.5)$$

$$\mathbf{h}_t^{\text{MLP}} = \phi(W^{(2)}\mathbf{z}_t^{(1)} + \mathbf{b}^{(2)}) \quad (4.6)$$

where $W^{(1)}, W^{(2)}$ and $\mathbf{b}^{(1)}, \mathbf{b}^{(2)}$ are learnable parameters, and $\phi(\cdot)$ denotes a nonlinear activation function. This formulation highlights the role of the MLP encoder as a nonlinear feature compressor over the full rolling state, but without an explicit mechanism for sequential memory or pairwise temporal interaction.

Thus, MLP-PPO serves as the simplest learned baseline for testing whether nonlinear compression of the rolling state is sufficient for portfolio allocation.

LSTM-PPO baseline.

For the *LSTM-PPO* baseline, the rolling state is processed sequentially along the temporal dimension. Let $\mathbf{x}_\tau$ denote the flattened asset-feature representation at lookback step τ , where $\tau = 1, 2, \dots, L$. The LSTM encoder updates its internal states through the standard gating mechanism:

$$\begin{aligned} \mathbf{i}_\tau &= \sigma(W_i\mathbf{x}_\tau + U_i\mathbf{h}_{\tau-1} + \mathbf{b}_i), \\ \mathbf{f}_\tau &= \sigma(W_f\mathbf{x}_\tau + U_f\mathbf{h}_{\tau-1} + \mathbf{b}_f), \\ \mathbf{o}_\tau &= \sigma(W_o\mathbf{x}_\tau + U_o\mathbf{h}_{\tau-1} + \mathbf{b}_o), \\ \tilde{\mathbf{c}}_\tau &= \tanh(W_c\mathbf{x}_\tau + U_c\mathbf{h}_{\tau-1} + \mathbf{b}_c), \\ \mathbf{c}_\tau &= \mathbf{f}_\tau \odot \mathbf{c}_{\tau-1} + \mathbf{i}_\tau \odot \tilde{\mathbf{c}}_\tau, \\ \mathbf{h}_\tau &= \mathbf{o}_\tau \odot \tanh(\mathbf{c}_\tau). \end{aligned} \quad (4.7)$$

where $\mathbf{i}_\tau$, $\mathbf{f}_\tau$, and $\mathbf{o}_\tau$ denote the input, forget, and output gates, respectively; $\mathbf{c}_\tau$ denotes the cell state; and $\mathbf{h}_\tau$ denotes the hidden state. The final hidden represen-

tation is then used as the latent market state:

$$\mathbf{h}_t^{\text{LSTM}} = \mathbf{h}_L. \quad (4.8)$$

This recurrent formulation enables the encoder to accumulate information across the lookback window through hidden-state transitions and gated memory updates [24, 28]. Compared with the MLP-PPO baseline, the LSTM-PPO model therefore captures temporal dependency more explicitly. However, unlike the Transformer encoder, it processes the historical window sequentially rather than through self-attention across all lookback positions [4, 5, 28].

Thus, LSTM-PPO provides a stronger sequential baseline than MLP-PPO because it explicitly models temporal dependency across the lookback window.

Common PPO-based policy block.

In both baselines, the latent representation $\mathbf{h}_t$ is passed into the same PPO-based actor-critic block used in the main model. The actor network transforms the latent state into portfolio scores and then into long-only, fully invested portfolio weights through a softmax mapping, while the critic estimates the value of the current state to support policy improvement. Therefore, the comparison among *MLP-PPO*, *LSTM-PPO*, and *Transformer-PPO* is intended to isolate the effect of encoder design while keeping the reinforcement learning framework fixed.

Taken together, these two baselines provide progressively stronger non-attention alternatives against which the contribution of the Transformer-based encoder can be assessed more clearly.

4.6 Training, Validation, and Test Protocol

All learned models are trained with Proximal Policy Optimization (PPO). In this thesis, PPO is kept as the shared optimization method for MLP-PPO, LSTM-PPO, and Transformer-PPO [3, 10, 11]. This is important for the comparison. If the optimizer changed across models, it would be harder to tell whether the result came from the encoder or from the reinforcement learning algorithm. By fixing PPO, the

experiment can focus more directly on the effect of state representation.

For each learned model, PPO is trained in the same general way. The model first collects portfolio trajectories from the training environment. These trajectories provide rewards, value estimates, and other information needed for policy improvement. The actor and critic are then updated over repeated optimization epochs. This training procedure is kept the same for MLP-PPO, LSTM-PPO, and Transformer-PPO, so that the comparison mainly reflects the quality of the state encoder rather than changes in the PPO workflow.

Training is performed only on the training subset. The validation subset is used for checkpoint selection and model comparison across epochs. The final reported out-of-sample performance is always based on the test subset. This protocol is applied consistently across the MLP-PPO, LSTM-PPO, and Transformer-PPO models, as well as across the reward-design comparison for the Transformer variant.

Model selection is based on validation performance rather than test performance. The checkpoint corresponding to the best validation result is retained and then evaluated on the test set. This separation helps preserve the validity of the out-of-sample evaluation and reduces the risk of implicit test-set overfitting [23, 24]. In this way, the empirical results reported later in the thesis can be interpreted as genuine out-of-sample evidence rather than as artifacts of repeated test-set tuning.

It is important to note that the trained model is not re-estimated or updated online during the test period. Portfolio weights are updated sequentially over time as new states are observed, but the model parameters remain fixed after validation-based checkpoint selection. This preserves the interpretation of the reported test results as genuine out-of-sample performance.

For clarity and reproducibility, the main implementation settings used in the learned-model experiments are summarized in Table 4.2.

Table 4.2: Main implementation settings used in the learned-model experiments

Component	Setting
Main data horizon	January 1, 2020 to December 31, 2024
Asset universe size	5 assets
Lookback length L	20 trading days
State representation	Rolling multi-asset tensor $X_t \in \mathbb{R}^{L \times N \times F}$
Portfolio setting	Long-only, fully invested
Learned models	MLP-PPO, LSTM-PPO, Transformer-PPO
Policy optimizer	Proximal Policy Optimization (PPO)
Training split	Chronological training subset
Validation split	Chronological validation subset
Test split	Chronological out-of-sample test subset
Checkpoint selection	Best validation performance
Reward (cost-aware)	$R_t = \log(G_t) - c\ w_t - w_{t-1}\ _1$
Reward (no-cost variant)	$R_t = \log(G_t)$
Portfolio growth term	$G_t = w_t^\top y_t$
Action constraint	$w_{t,i} \geq 0, \sum_{i=1}^N w_{t,i} = 1$
Main explainability output	Temporal attention heatmaps and grouped averages

These settings define the common empirical framework used for the learned-model comparison. Keeping the main environment, state construction, portfolio constraints, and evaluation protocol fixed is important so that the comparison remains focused on encoder design and reward specification rather than on unrelated implementation differences.

4.7 Reward Design and Transaction Cost Modeling

The reward function is designed to reflect realized portfolio outcome while penalizing excessive rebalancing under transaction costs [10, 11, 18]. Let y_t denote the

vector of realized gross asset returns at time t . The one-step gross portfolio growth is defined as

$$G_t = w_t^\top y_t. \quad (4.9)$$

The cost-aware reward is then written as

$$R_t = \log(G_t) - c\|w_t - w_{t-1}\|_1, \quad (4.10)$$

where $c \geq 0$ is the transaction cost coefficient and $\|w_t - w_{t-1}\|_1$ measures the turnover induced by rebalancing. Substituting the definition of G_t gives

$$R_t = \log(w_t^\top y_t) - c\|w_t - w_{t-1}\|_1. \quad (4.11)$$

In implementation, the environment may equivalently use simple returns r_t , in which case the portfolio growth term is written as $1 + w_t^\top r_t$. Under the fully invested portfolio constraint, these two formulations are equivalent because

$$w_t^\top y_t = 1 + w_t^\top r_t. \quad (4.12)$$

Therefore, the implementation remains fully consistent with the theoretical formulation adopted in the methodology [6, 10, 11].

To support RQ2, the thesis compares two Transformer training variants:

- a **cost-aware** training variant, in which the reward includes the transaction cost penalty;
- a **no-cost** training variant, in which the reward is defined without the transaction cost penalty during training.

The purpose of this comparison is not limited to identifying differences in cumulative return. More importantly, it is used to examine how reward design influences rebalancing behavior, turnover, trading cost exposure, and the broader trading profile of the learned policy. This makes the reward-design analysis directly relevant to the practical interpretation of reinforcement learning-based portfolio management.

4.8 Evaluation Metrics

The evaluation does not rely only on the final portfolio return. A strategy may end with a good cumulative return, but still be risky if it goes through large drawdowns, trades too often, or loses too much value through transaction costs. For that reason, the empirical analysis uses several metrics to describe both performance and trading behavior.

The main metrics reported are:

- Final Portfolio Value;
- Cumulative Return;
- Average Daily Return;
- Daily Volatility;
- Sharpe Approximation;
- Maximum Drawdown;
- Average Turnover;
- Total Trading Cost;
- Average Trading Cost.

During evaluation, V_t is used for the portfolio value at time t , and r_t^p is used for the realized simple return of the portfolio. The main metrics used in the result

discussion are defined below:

$$\text{Cumulative Return: CR} = V_T - 1, \quad (4.13)$$

$$\text{Sharpe Approximation: SR} = \frac{\mu(r^p)}{\sigma(r^p)} \sqrt{252}, \quad (4.14)$$

$$\text{Maximum Drawdown: MDD} = \min_t \left(\frac{V_t}{\max_{s \leq t} V_s} - 1 \right), \quad (4.15)$$

$$\text{Average Turnover: TO}_{avg} = \frac{1}{T} \sum_{t=1}^T \|w_t - w_{t-1}\|_1, \quad (4.16)$$

$$\text{Total Trading Cost: TC}_{total} = \sum_{t=1}^T c \|w_t - w_{t-1}\|_1. \quad (4.17)$$

where $\mu(r^p)$ and $\sigma(r^p)$ are the mean and standard deviation of portfolio simple returns during the evaluation period.

The metrics are not meant to be read separately. Final value and cumulative return are useful starting points because they show the ending wealth level, but they do not describe the full investment experience. For example, two policies may end close to each other in cumulative return, while one of them suffers a much larger loss before recovering. This is why maximum drawdown is reported together with return. The Sharpe approximation is also included to give a simple indication of how much return is obtained relative to return volatility [23, 24].

For the reward-design experiment, turnover and trading cost are especially important. A learned policy may increase return by adjusting the portfolio more often, but frequent rebalancing can also make the strategy less attractive after costs. Therefore, in Chapter 5, the result is interpreted by looking at return, drawdown, turnover, and cost at the same time, rather than by ranking strategies only by cumulative return.

When the metrics are reported across different data subsets, validation results are used mainly for model selection, while test results are used for the final out-of-sample comparison. This keeps the empirical interpretation aligned with the research questions and avoids drawing conclusions from training-period behavior.

4.9 Attention-Based Explainability Protocol

To support the explainability objective of the thesis, an attention-based interpretability analysis is conducted for the trained Transformer model [5, 14, 19, 20]. Since the Transformer encoder in the final implementation operates primarily over the temporal dimension after each time step is represented as a flattened asset-feature slice, the extracted attention maps are interpreted mainly as temporal attention maps across lookback positions.

The explainability analysis includes two complementary forms. First, single-window attention heatmaps are extracted for representative out-of-sample test cases. These visualizations are used to examine whether the encoder distributes attention uniformly across the historical window or instead emphasizes selected lookback positions.

Second, the analysis also uses average attention maps over groups of test windows. These groups include random windows, high-volatility windows, and low-volatility windows. Looking at grouped maps helps avoid relying too much on a few selected examples, and gives a broader picture of how temporal attention changes across different market conditions.

The attention maps are not used as full causal explanations of the model’s decisions. They are better understood as a diagnostic view of how the Transformer encoder uses historical information before the actor produces portfolio weights. For this reason, the attention analysis is included as supplementary explainability evidence, not as another performance metric.

Algorithm 2 summarizes the attention-based explainability procedure used in the thesis.

Algorithm 2 Attention-Based Explainability Procedure

- 1: **Input:** Trained Transformer-based policy and out-of-sample test windows $\{X_t\}$
- 2: **Output:** Attention heatmaps and grouped attention summaries
- 3: **for** each selected test window X_t **do**
- 4: Pass X_t through the Transformer encoder
- 5: Extract the corresponding attention maps A_t
- 6: Aggregate attention across heads or layers if needed
- 7: Visualize the resulting temporal attention heatmap
- 8: **end for**
- 9: **for** each analysis group $\mathcal{G}$ **do**
- 10: Compute the average attention map:

$$\bar{A}_{\mathcal{G}} = \frac{1}{|\mathcal{G}|} \sum_{t \in \mathcal{G}} A_t$$

- 11: Compute summary statistics and save grouped visualizations
 - 12: **end for**
-

This procedure helps show how the trained Transformer encoder places attention across the historical lookback window in different out-of-sample cases.

4.10 Reliability and Fairness of Evaluation

To make the learned-model comparison consistent, the three PPO-based models are run under the same conditions. MLP-PPO, LSTM-PPO, and Transformer-PPO use the same asset universe, chronological data split, feature pipeline, 20-day lookback window, portfolio constraints, reward function, PPO training procedure, and evaluation metrics. This setup makes the comparison mainly about the encoder, rather than about differences in data processing or training design.

The benchmark strategies are included for a different reason. They provide simple reference points for judging whether the learned policies offer value beyond basic

allocation rules. This is important because a learned model may outperform other learned models, but still remain weaker than passive strategies that are transparent, stable, and easy to implement.

The reward-design comparison is also kept consistent. The cost-aware and no-cost Transformer variants are evaluated on the same test period and reported with the same metrics, including return, drawdown, turnover, and trading cost. This makes it possible to discuss not only final portfolio value, but also how the reward specification changes the behavior of the learned policy.

Finally, model selection is based on validation performance only, while the test set is reserved for final out-of-sample evaluation. This separation reduces the risk of implicit test-set overfitting and supports a more reliable interpretation of the reported results [23, 24].

This design also reduces the risk that the reported conclusions are driven by a single favorable metric, because the final interpretation considers return, drawdown, turnover, trading cost, benchmark comparison, and attention-based diagnostic evidence together.

Overall, the empirical design is aligned with the research questions through controlled comparisons. RQ1 is examined by comparing Transformer-PPO, LSTM-PPO, and MLP-PPO under the same state construction, PPO framework, portfolio constraints, and evaluation metrics. RQ2 is examined by comparing the cost-aware and no-cost Transformer variants under the same test setting. The attention-based analysis is treated as supporting interpretability evidence, while the main conclusions are drawn from portfolio-level performance, drawdown, turnover, and transaction-cost exposure.

4.11 Chapter Summary

This chapter described the experimental design used to evaluate the proposed framework. The main setting uses a controlled five-asset portfolio over the 2020–2024 period, with chronological training, validation, and test splits. The experiments compare passive benchmarks, MLP-PPO, LSTM-PPO, and Transformer-PPO under

a common evaluation protocol.

The chapter also defined the reward-design comparison, evaluation metrics, attention-analysis procedure, and fairness controls used in the empirical study. The next chapter reports the out-of-sample results and discusses their implications for the research questions.

Chapter 5

EMPIRICAL RESULTS AND DISCUSSION

5.1 Chapter Overview

This chapter presents and discusses the empirical results of the proposed Transformer-based deep reinforcement learning framework for dynamic portfolio allocation. The analysis follows the experimental design described in Chapter 4 and focuses on the controlled five-year setting from January 1, 2020 to December 31, 2024. The chapter does more than list the test results. It also discusses what those results imply for the main parts of the framework: the market representation, the reward design, the rebalancing behavior, and the position of the learned policies relative to simple benchmark strategies.

The discussion begins with the overall out-of-sample performance of all strategies, including both benchmark rules and learned PPO-based models. This gives a general picture of where the proposed model stands. The chapter then focuses on the learned-model comparison by examining Transformer-PPO against MLP-PPO and LSTM-PPO under the same PPO setting. After that, the reward-design experiment compares the cost-aware and no-cost Transformer variants to show how transaction-cost-aware training affects the learned policy. The chapter also includes a temporal attention analysis as a supporting interpretability component. The 20-day lookback window is treated as a controlled design choice used consistently across

the learned-model comparison. The final discussion integrates the main empirical findings and clarifies the scope of the conclusions.

The results are discussed carefully because portfolio quality cannot be judged by cumulative return alone. In a dynamic allocation setting, a strategy must also be examined through risk-adjusted performance, drawdown, turnover, and trading-cost exposure. This is especially important for reinforcement-learning-based portfolio models, since the reward function and the rebalancing mechanism directly influence the type of trading behavior that the policy learns. Therefore, the results in this chapter are interpreted through a portfolio-management perspective rather than as a pure prediction-performance comparison.

5.2 Overall Out-of-Sample Portfolio Performance

5.2.1 Main Test-Set Performance Comparison

Table 5.1 reports the main five-year test results for the benchmark strategies and learned models. The comparison includes three benchmark strategies, namely Equal Weight, Buy-and-Hold Equal Weight, and Random Policy, together with three PPO-based learned models, namely Transformer-PPO, LSTM-PPO, and MLP-PPO. All reported results are based on the out-of-sample test set, after model selection has been performed using the validation set.

Table 5.1: Main out-of-sample performance comparison between benchmark strategies and learned PPO-based allocation policies in the five-year setting.

Model	Cum. Return	Sharpe Approx.	MDD	Avg. Turnover	Trading Cost	Rank
Equal Weight	0.2501	1.7390	-0.0888	0.0000	0.0000	1
Buy-and-Hold	0.2492	1.6802	-0.0967	0.0000	0.0000	2
Transformer-PPO	0.1947	1.5747	-0.1052	0.2165	0.0544	3
LSTM-PPO	0.1692	1.5377	-0.0911	0.1913	0.0480	4
MLP-PPO	0.1212	1.4207	-0.0968	0.2679	0.0672	5
Random Policy	0.0008	1.1497	-0.1049	0.6311	0.1584	6

The overall ranking shows that the passive benchmark strategies remain the strongest performers in the main five-year setting. Equal Weight achieves the highest cumulative return of 0.2501 and the highest Sharpe approximation of 1.7390. Buy-and-Hold Equal Weight follows closely, with a cumulative return of 0.2492 and a Sharpe approximation of 1.6802. These two benchmark strategies therefore outperform all learned models in terms of cumulative return in the main test period.

Within the learned-model group, Transformer-PPO gives the best result. It reaches a cumulative return of 0.1947, compared with 0.1692 for LSTM-PPO and 0.1212 for MLP-PPO. The same pattern appears in the Sharpe approximation, where Transformer-PPO records 1.5747, higher than 1.5377 for LSTM-PPO and 1.4207 for MLP-PPO. In this comparison, the Transformer encoder therefore appears to provide the most useful state representation for the PPO policy.

The Random Policy sits at the other end of the table. Its cumulative return is almost zero, at 0.0008, and it also produces much higher turnover and trading cost than the other strategies. This makes it a useful lower reference point. The difference between Random Policy and the learned models suggests that the PPO-based policies are not just generating random allocation weights. They are learning some useful structure from the sequential portfolio environment.

The same table also helps keep the interpretation careful. Transformer-PPO is the strongest learned model, but it still does not beat every benchmark. Equal Weight and Buy-and-Hold Equal Weight remain strong during the test period because they provide diversified exposure, avoid model-estimation risk, and do not create trading cost in the reported setting. For this reason, Transformer-PPO should be interpreted as the best learned policy in the controlled DRL comparison, not as a strategy that dominates all portfolio benchmarks.

5.2.2 Portfolio Value Dynamics

Figure 5.1 presents the full benchmark-versus-model portfolio value curves over the test period. The figure helps visualize the numerical results reported in Table 5.1. It shows that Equal Weight and Buy-and-Hold Equal Weight remain above the learned models for most of the out-of-sample horizon. This confirms that the passive

benchmarks are not only strong at the final evaluation point, but also maintain relatively favorable portfolio trajectories throughout the test period.

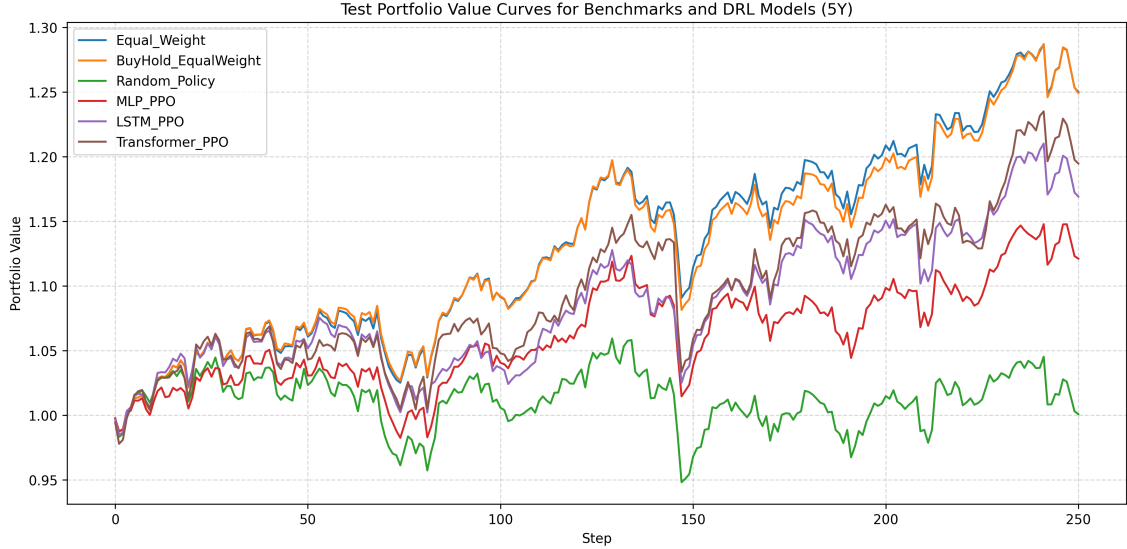


Figure 5.1: Full out-of-sample portfolio value curves for benchmark strategies and PPO-based learned allocation models over the five-year test period.

The learned policies still show meaningful differences among themselves. Transformer-PPO generally produces a stronger portfolio path than MLP-PPO and remains competitive relative to LSTM-PPO over the test horizon. This visual pattern is consistent with the numerical ranking in Table 5.1, where Transformer-PPO achieves the highest cumulative return and Sharpe approximation among the learned models.

The portfolio curves also make the benchmark result easier to interpret. Transformer-PPO performs well compared with the other learned policies, but the passive portfolios remain above it for most of the test period. This means that the learned model is not failing in the DRL comparison; rather, it is being compared against very strong and low-cost benchmark rules. In this five-asset universe, a simple diversified position already captures a large part of the upward movement in the test period. This helps explain why the passive rules remain difficult for the learned policies to beat, even though the learned models are not performing randomly.

The result is therefore best interpreted at two levels. Within the learned-model group, Transformer-PPO is the strongest of the three encoder variants. In the broader portfolio-strategy comparison, however, Equal Weight and Buy-and-Hold

Equal Weight remain stronger reference strategies. This distinction is important because the experiment supports the value of Transformer-based encoding for learned dynamic allocation, but it does not show that DRL dominates passive allocation in this setting.

5.2.3 Practical Interpretation of Benchmark Results

The strong performance of the passive benchmarks should be interpreted carefully. Equal Weight and Buy-and-Hold Equal Weight are simple, but they are not weak baselines in this setting. The five selected assets are large U.S. stocks, so a fixed diversified exposure can already perform well during much of the test period. These benchmarks also do not need to estimate model parameters, and in the reported comparison they do not create additional trading cost.

The learned PPO policies face a different problem. They have to learn a mapping from historical market information to portfolio weights. This can be affected by limited training data, noisy market movements, and estimation error. For this reason, a learned model may not always beat a simple passive allocation, especially when the passive allocation benefits from broad upward market exposure.

However, the learned models still have value in this thesis because they are adaptive. Unlike Equal Weight and Buy-and-Hold Equal Weight, they update portfolio weights according to the encoded market state rather than following a fixed allocation rule. Therefore, the result should be read in two levels. In the full benchmark comparison, the passive strategies remain stronger. Within the learned-model group, Transformer-PPO gives the best result, which supports the role of Transformer-based state encoding in improving the PPO allocation policy.

Therefore, the benchmark result should be interpreted as an important empirical finding rather than as a failure of the proposed framework. It shows that, in a small and strongly performing equity universe, adaptive learned policies must be evaluated against simple low-cost allocation rules. This strengthens the validity of the empirical study because the proposed model is not only compared with weaker learned baselines, but also with practical benchmark strategies that are difficult to outperform.

5.3 Effect of Market Representation on Learned Policies

This section focuses on the effect of market representation on learned portfolio policies. While the previous section considered the full comparison across both benchmarks and learned models, the present section narrows the analysis to the three PPO-based learned policies. This comparison is central to the thesis because all three learned models share the same portfolio environment, state-construction protocol, action constraints, reward formulation, and PPO-based optimization framework. The main difference among them lies in the encoder used to transform the rolling historical market window into a latent state representation.

The three learned encoders represent different levels of temporal modeling capacity. The MLP encoder provides a simple nonlinear compression of the flattened rolling state. The LSTM encoder introduces recurrent temporal processing through hidden-state updates. The Transformer encoder, in contrast, uses self-attention to construct a contextual representation across the lookback window. Therefore, the comparison among MLP-PPO, LSTM-PPO, and Transformer-PPO helps assess whether attention-based market representation contributes positively to dynamic portfolio allocation under the common PPO framework.

5.3.1 Learned Encoder Comparison under PPO

The learned-model comparison shows a clear ordering in the main five-year test setting. As reported in Table 5.1, Transformer-PPO achieves the highest cumulative return among the learned models, followed by LSTM-PPO and then MLP-PPO. The cumulative return of Transformer-PPO is 0.1947, compared with 0.1692 for LSTM-PPO and 0.1212 for MLP-PPO.

The same ordering is observed for the Sharpe approximation. Transformer-PPO records a Sharpe approximation of 1.5747, while LSTM-PPO and MLP-PPO record 1.5377 and 1.4207, respectively. Although the difference between Transformer-PPO and LSTM-PPO is not extremely large, the Transformer-based policy remains ahead in both cumulative return and risk-adjusted performance within the learned-model

group.

Figure 5.2 provides a visual comparison of the learned-model-only portfolio value curves. This figure removes the passive benchmarks from the plot so that the differences among the learned policies can be inspected more clearly.

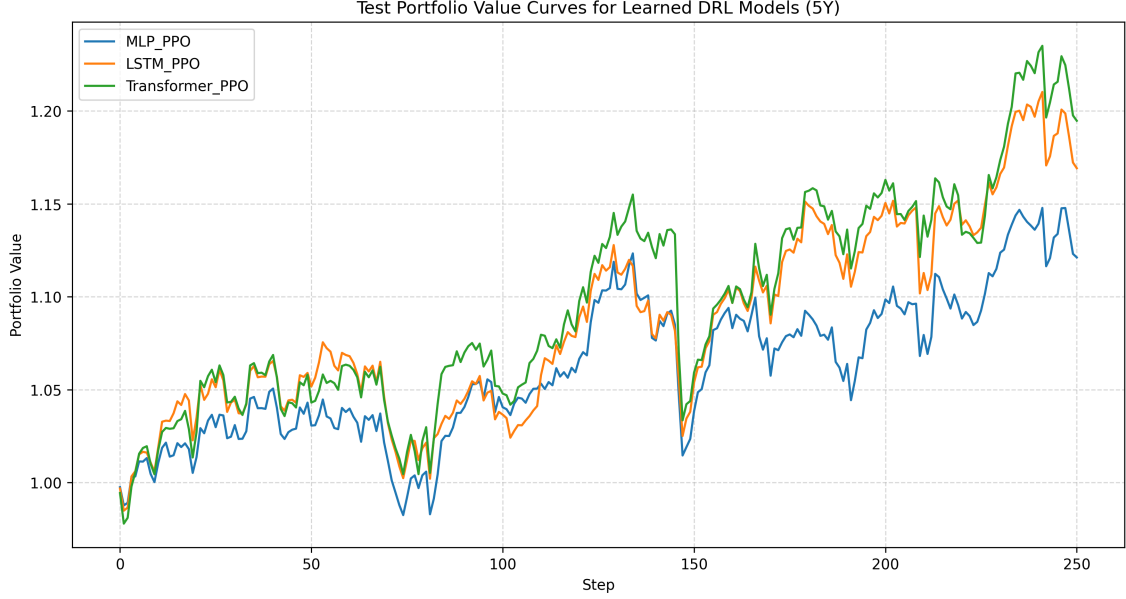


Figure 5.2: Out-of-sample portfolio value curves for learned PPO-based allocation models over the five-year test period.

The learned-model curves show that Transformer-PPO generally maintains a stronger portfolio path than MLP-PPO and remains competitive relative to LSTM-PPO throughout the test period. The MLP-PPO curve is weaker overall, which suggests that a simple feedforward compression of the rolling historical window is less effective for this dynamic allocation task. The LSTM-PPO curve performs better than MLP-PPO, reflecting the benefit of explicit sequential modeling. However, the Transformer-PPO model still achieves the strongest final learned-model performance, which supports the use of self-attention as a market representation mechanism in this setting.

These findings are consistent with the methodological design of the thesis. The purpose of the Transformer encoder is not to directly forecast individual asset prices, but to construct a latent market state that is useful for portfolio decision-making. The empirical results suggest that this representation is more effective for the down-

stream PPO policy than the representations produced by the simpler MLP and LSTM encoders.

5.3.2 Transformer-Based Representation versus MLP and LSTM Encoders

The difference among the learned encoder variants can be interpreted through their representation mechanisms. The MLP-PPO model receives the rolling historical state but treats it primarily as a flattened input. Although the MLP can learn nonlinear transformations, it does not explicitly preserve the temporal structure of the lookback window. As a result, it may be less effective at distinguishing which historical positions are more relevant for the current allocation decision.

The LSTM-PPO model improves on this limitation by processing the historical window sequentially. Through its recurrent structure, it can accumulate information over time and retain parts of the past through gated memory. This makes LSTM-PPO a stronger temporal baseline than MLP-PPO. The empirical results support this view, since LSTM-PPO outperforms MLP-PPO in cumulative return and Sharpe approximation.

The Transformer-PPO model adopts a different representation strategy. Instead of compressing the input through a purely feedforward mapping or processing it only through recurrent transitions, the Transformer encoder uses self-attention across the lookback window. This allows the model to assign different levels of emphasis to different historical positions when constructing the latent state representation. In a portfolio setting, this is useful because relevant market information may not always be located at the most recent time step. Some patterns may become meaningful only when the model compares different points within the historical window.

This ordering among the learned models supports the main argument of the thesis about market representation. The Transformer-based encoder appears to provide a more useful state representation for the PPO policy than the MLP and LSTM alternatives under the controlled five-year setting. However, this conclusion should remain within the scope of the experiment. The results show an improvement over

the selected learned encoder baselines, not a universal superiority of Transformer models across all possible markets, asset universes, or DRL configurations.

5.3.3 Risk and Turnover Profile of Learned Models

Although Transformer-PPO achieves the strongest return-oriented performance among the learned models, its risk and turnover profile requires a more careful interpretation. In Table 5.1, Transformer-PPO has a maximum drawdown of -0.1052, which is deeper than the LSTM-PPO drawdown of -0.0911 and also slightly deeper than the MLP-PPO drawdown of -0.0968. This means that the Transformer-based policy is not the most conservative learned model in terms of downside path risk.

The turnover metrics show a similar trade-off. Transformer-PPO has an average turnover of 0.2165 and total trading cost of 0.0544. These values are lower than those of MLP-PPO, which has average turnover of 0.2679 and total trading cost of 0.0672, but higher than those of LSTM-PPO, which has average turnover of 0.1913 and total trading cost of 0.0480. Therefore, the Transformer-based policy improves learned portfolio growth and Sharpe approximation, but it does not minimize turnover or trading cost among all learned models.

This observation is important because it keeps the result in context. The Transformer encoder improves the learned policy’s overall performance profile, but the improvement comes with a trade-off. Transformer-PPO is better at generating portfolio growth than the simpler learned baselines, but the result is not one-sided. Compared with LSTM-PPO, it can still have a deeper drawdown and higher turnover. In portfolio evaluation, this difference matters. A strategy with a higher return is not always preferable if it also exposes the investor to larger temporary losses or more frequent rebalancing.

Even with this caution, the result still supports the main argument of the thesis. The key question is whether Transformer-based state encoding improves the learned allocation policy under the same PPO framework. Based on the learned-model comparison, the answer is yes. However, the risk and turnover results show that a better representation does not automatically make the whole portfolio profile better

in every respect.

Overall, the learned-model results suggest that the Transformer encoder is useful in the controlled empirical setting. It improves cumulative return and Sharpe approximation relative to MLP-PPO and LSTM-PPO, while the drawdown and turnover results remind us that the conclusion should remain balanced. This leads naturally to the next section, where the analysis turns to reward design and its effect on the Transformer-based policy.

5.4 Effect of Transaction-Cost-Aware Reward Design

This section examines how reward design affects the behavior and performance of the Transformer-based portfolio policy. In the previous section, the Transformer encoder was shown to produce the strongest learned-model performance under the common PPO framework. However, portfolio performance is not determined by representation quality alone. In reinforcement learning, the reward function also plays a central role because it defines what type of behavior the agent is encouraged to learn.

To study this effect, the thesis compares two Transformer training variants. The first variant is trained with the cost-aware reward function, where transaction cost is included as a penalty on portfolio rebalancing. The second variant is trained with a no-cost reward, where the reward only reflects portfolio growth during training. Both variants are then evaluated under the same reporting framework so that the effect of reward specification can be examined through cumulative return, Sharpe approximation, drawdown, turnover, trading cost, and portfolio allocation behavior.

This comparison is important because transaction costs are not merely an external reporting item in dynamic portfolio allocation. They influence whether a learned policy is practically meaningful. A policy that performs well before trading frictions may become less convincing once repeated rebalancing is penalized. Therefore, the purpose of this section is not only to compare final return, but also to examine how reward design changes the broader return–risk–turnover profile of the learned policy.

5.4.1 Cost-Aware and No-Cost Transformer Variants

Table 5.2 reports the focused comparison between the cost-aware-trained and no-cost-trained Transformer variants. The table includes two evaluation modes. The first is the realistic cost-aware evaluation, where trading costs are applied to the portfolio outcome. The second is the frictionless no-cost evaluation, where transaction costs are removed in order to inspect the raw portfolio growth potential of each trained policy.

Table 5.2: Performance comparison between cost-aware and no-cost Transformer-PPO variants under realistic and frictionless evaluation settings.

Metric	Cost-Aware Realistic	No-Cost Realistic	Cost-Aware Frictionless	No-Cost Frictionless
Split	test	test	test	test
Steps	251	251	251	251
Final Portfolio Value	1.1947	1.1898	1.2615	1.2444
Cumulative Return	0.1947	0.1898	0.2615	0.2444
Avg. Daily Return	0.0010	0.0009	0.0010	0.0009
Daily Volatility	0.0098	0.0077	0.0098	0.0077
Sharpe Approx.	1.5747	1.8691	1.5747	1.8691
Max Drawdown	-0.1052	-0.0614	-0.1008	-0.0585
Avg. Turnover	0.2165	0.1790	0.2165	0.1790
Total Trading Cost	0.0544	0.0449	0.0000	0.0000
Avg. Trading Cost	0.000217	0.000179	0.000000	0.000000

Under realistic cost-aware evaluation, the cost-aware-trained Transformer achieves a final portfolio value of 1.1947 and a cumulative return of 0.1947. The no-cost-trained Transformer achieves a final portfolio value of 1.1898 and a cumulative return of 0.1898. The difference in final return is therefore small, but the cost-aware-trained model finishes slightly ahead in cumulative return.

However, the no-cost-trained model records a stronger Sharpe approximation. Its Sharpe approximation is 1.8691, compared with 1.5747 for the cost-aware-trained model. The no-cost-trained model also has a shallower maximum drawdown of

-0.0614, while the cost-aware-trained model has a deeper maximum drawdown of -0.1052. These results show that the cost-aware reward does not produce a uniformly better policy across all metrics.

The turnover and cost metrics show the same issue. Under realistic evaluation, the no-cost-trained model has an average turnover of 0.1790 and total trading cost of 0.0449. The cost-aware-trained model has a higher average turnover of 0.2165 and higher total trading cost of 0.0544. This result is somewhat counterintuitive because one might expect the cost-aware-trained model to trade less aggressively. In the present experiment, however, the cost-aware training objective does not automatically lead to lower turnover in the test period.

Under frictionless no-cost evaluation, both models produce higher cumulative returns because transaction costs are removed. The cost-aware-trained variant reaches a cumulative return of 0.2615, compared with 0.2444 for the no-cost-trained variant. This means that, before transaction costs are deducted, the cost-aware policy still produces slightly stronger portfolio growth over the test period. However, this advantage comes together with higher turnover, higher trading cost, and a deeper drawdown under realistic evaluation.

Overall, the comparison shows that reward design changes the learned policy in a meaningful way, but the effect is not straightforward. Adding transaction cost to the training reward changes the learned policy profile, but it does not guarantee improvement in every reported metric.

5.4.2 Return–Risk–Turnover Trade-off

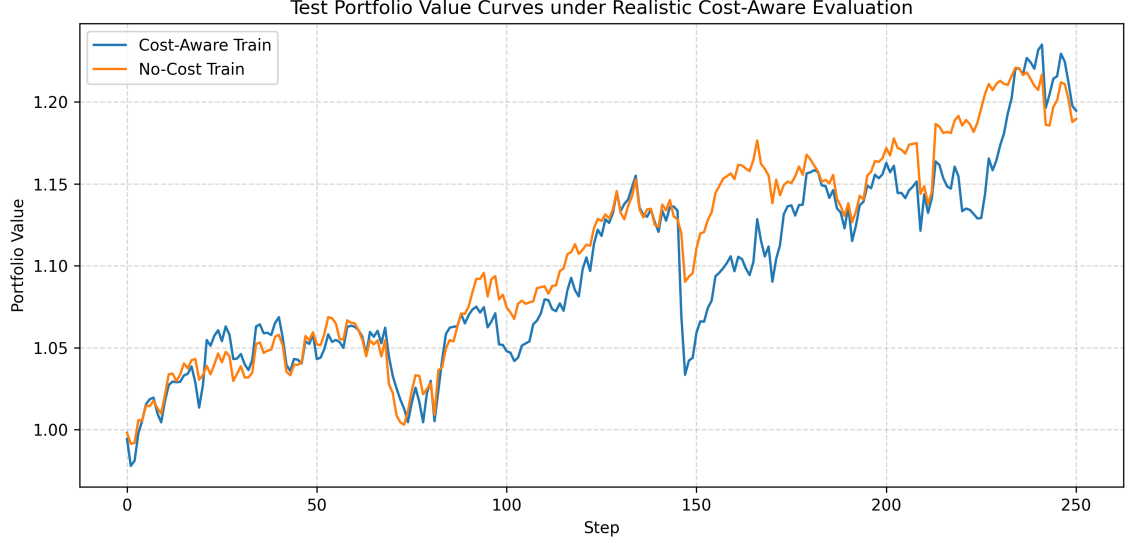


Figure 5.3: Out-of-sample portfolio value curves of cost-aware and no-cost Transformer-PPO variants under realistic transaction-cost-aware evaluation.

Figure 5.3 shows that the two Transformer variants follow broadly similar portfolio-value paths under realistic evaluation, with only a small final difference between the cost-aware-trained and no-cost-trained policies.

The cost-aware-trained model ends slightly higher, which is consistent with Table 5.2. However, the distance between the two curves is small. Therefore, the final cumulative return alone is not enough to judge the reward design.

The more useful comparison comes from looking at the full risk and trading profile. The cost-aware-trained policy improves final return slightly, but its path is not smoother. This becomes clearer in the drawdown comparison.

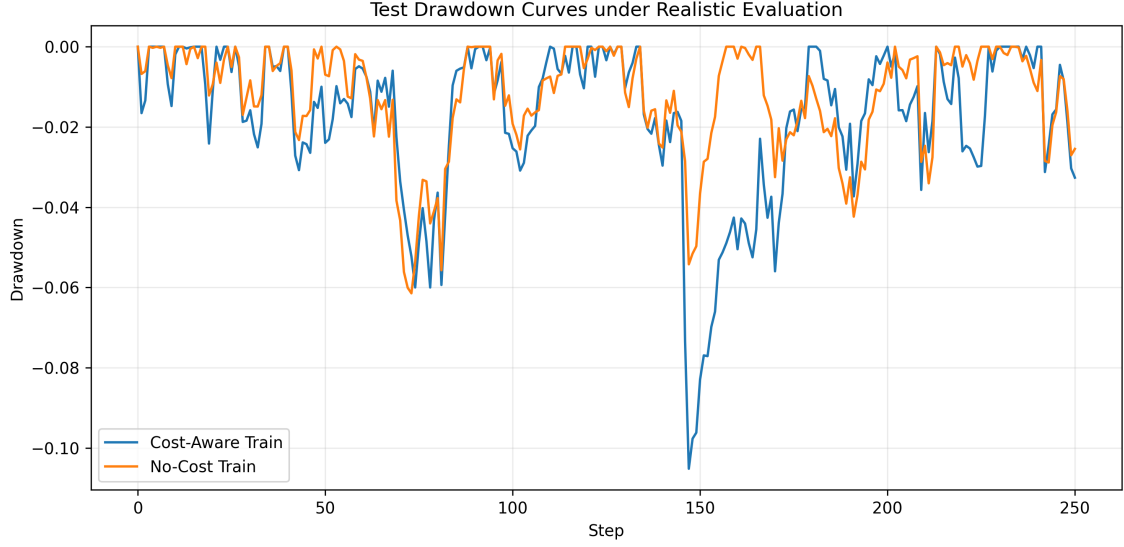


Figure 5.4: Drawdown comparison between cost-aware and no-cost Transformer-PPO variants under realistic transaction-cost-aware evaluation.

Figure 5.4 shows that the no-cost-trained policy has the smaller worst loss during the test period. Based on Table 5.2, its maximum drawdown is -0.0614 , compared with -0.1052 for the cost-aware-trained policy. This is an important difference because the cost-aware model has the slightly higher final return, but it also experiences the deeper temporary decline.

The turnover numbers point in the same direction. The cost-aware-trained model does not trade less in the test period. Its average turnover is 0.2165 , compared with 0.1790 for the no-cost-trained model. Its total trading cost is also higher, 0.0544 compared with 0.0449 . In this experiment, adding transaction cost to the reward changes the learned behavior, but the realized policy is not automatically more conservative.

The reward-design result is therefore mixed. The cost-aware-trained model gives a small gain in cumulative return, while the no-cost-trained model gives the better Sharpe approximation, shallower drawdown, lower turnover, and lower trading cost. For RQ2, the evidence shows that transaction-cost-aware reward design affects the policy, but its effect must be judged across return, risk, turnover, and cost together.

5.4.3 Portfolio Allocation Behavior under Different Reward Designs

Beyond portfolio value and risk metrics, the two reward designs also lead to different average allocation profiles. Figure 5.5 compares the average test-period portfolio weights produced by the cost-aware-trained and no-cost-trained Transformer variants. This figure is useful because it shows that reward design changes the composition of the learned portfolio, not only its final performance statistics.

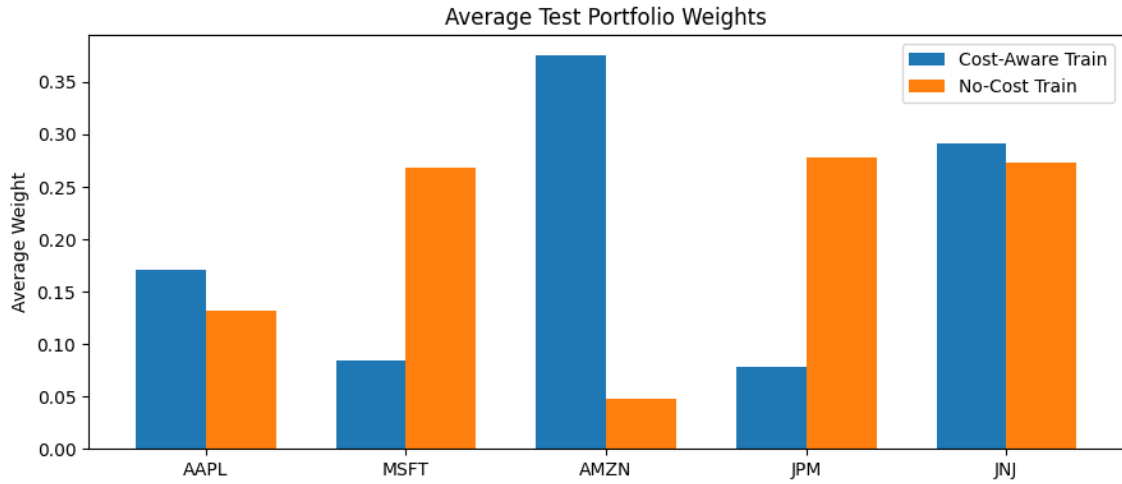


Figure 5.5: Average test portfolio weights of cost-aware and no-cost Transformer-PPO variants under realistic evaluation.

Figure 5.5 shows that the two reward designs lead to visibly different allocation profiles. The cost-aware-trained policy places more average weight on AMZN and JNJ, while the no-cost-trained policy assigns more weight to MSFT and JPM. Since both variants use the same Transformer architecture and PPO framework, the difference is mainly linked to the reward specification used during training.

This result is useful because it shows that reward design changes more than the final performance table. In reinforcement learning, the reward function shapes the behavior that the agent learns. Here, the transaction-cost-aware reward does not simply make the policy more passive. It leads the model to hold a different asset mix, and this difference is then reflected in the return, drawdown, turnover, and trading-cost results.

The answer to the second research question is therefore not one-sided. Transaction-

cost-aware reward design does affect both performance and policy behavior, but it does not improve every metric. In the reported test setting, it improves cumulative return slightly, while the no-cost-trained policy remains stronger on several risk and trading-cost measures.

5.5 Temporal Attention and Interpretability Analysis

This section presents the temporal attention analysis of the trained Transformer-PPO model. Since the proposed framework uses a Transformer encoder as the market representation module, the attention weights provide a supplementary way to inspect how the encoder distributes emphasis across the historical lookback window before the actor network generates portfolio weights.

This analysis is used for interpretability rather than for performance comparison. The attention maps are not used to claim that the model has a complete causal explanation for each portfolio decision. Instead, they are used as diagnostic evidence about the internal structure of the learned representation. In the implementation of this thesis, the Transformer encoder processes the historical input primarily across lookback positions after each time step is represented as a flattened asset-feature slice. Therefore, the attention maps are interpreted mainly as temporal attention maps rather than as direct asset-to-asset attribution maps.

This distinction is important. A temporal attention map can show which positions in the historical window receive relatively greater representational emphasis, but it does not directly prove why a particular asset receives a particular portfolio weight. For that reason, the attention analysis is treated as a supporting explainability component. It complements the performance results by showing whether the Transformer encoder learns structured temporal emphasis, while the main empirical conclusions remain based on portfolio performance, risk, turnover, and reward-design behavior.

5.5.1 Representative Attention Heatmaps

Figure 5.6 presents representative temporal attention heatmaps extracted from the trained Transformer encoder during the out-of-sample test period. Each heatmap visualizes how attention is distributed across the 20-step historical lookback window for a selected test sample. The horizontal axis represents source positions in the lookback window, while the vertical axis represents query positions. Brighter regions indicate stronger attention weight.

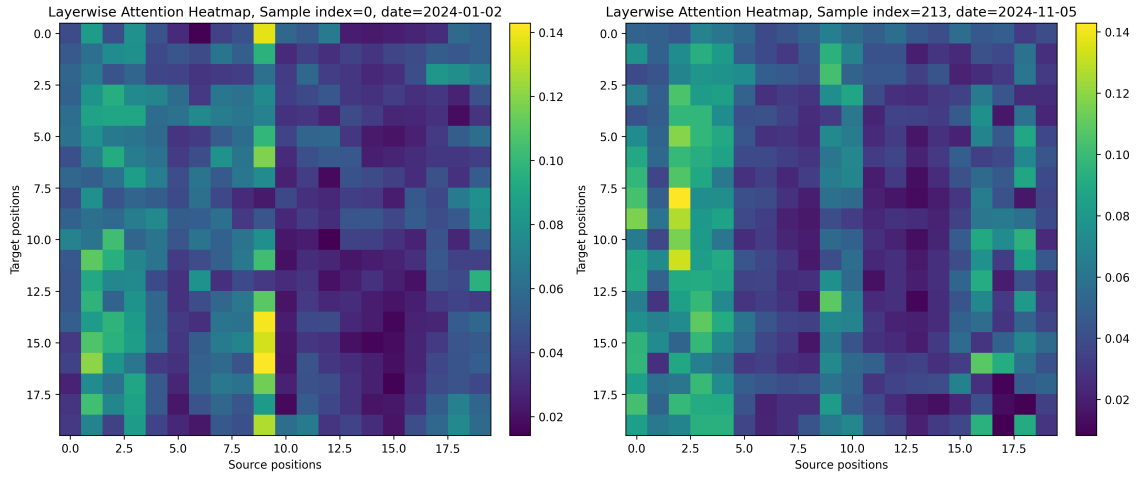


Figure 5.6: Representative layerwise temporal attention heatmaps from the Transformer-PPO model over the five-year test period.

The representative heatmaps show that attention is not distributed uniformly across all lookback positions. In the first selected test case, corresponding to an early out-of-sample window, the attention pattern contains visible concentration around selected interior source positions. In the second selected case, taken from a later out-of-sample window, the attention structure is again nonuniform, but the emphasized positions are different. This suggests that the trained Transformer encoder does not simply apply a fixed temporal template to every input.

The nonuniform structure is important because it indicates that the encoder is using the historical window selectively when forming the latent market representation. If the attention maps were almost flat across all positions, the interpretability value of the Transformer encoder would be weaker. Instead, the observed heatmaps suggest that the model assigns different levels of representational emphasis to dif-

ferent historical positions depending on the local market context.

These heatmaps still need to be read with caution. They are helpful examples of how the encoder behaves in selected windows, but a few individual cases are not enough to represent the whole test period. For that reason, the analysis is extended in the next subsection by using grouped average attention maps from different sets of test windows.

5.5.2 Grouped Attention Patterns under Different Market Conditions

To reduce the risk of over-interpreting a small number of individual heatmaps, the analysis also examines grouped average attention patterns. Figure 5.7 reports average temporal attention maps for random test windows, high-volatility windows, and low-volatility windows, together with a difference map between the high-volatility and low-volatility groups. This grouped analysis provides a broader view of how the Transformer encoder distributes attention under different market conditions.

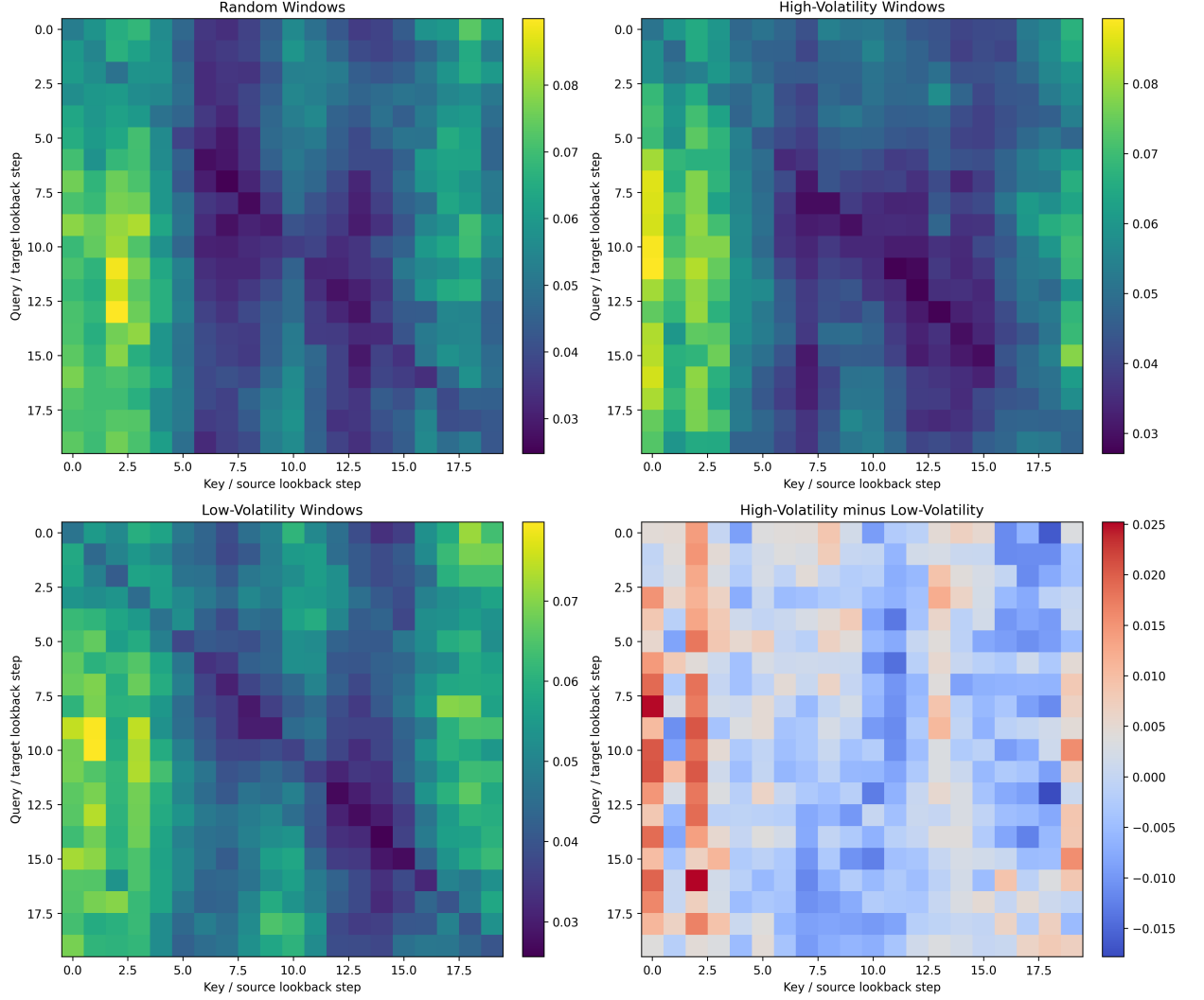


Figure 5.7: Grouped temporal attention analysis of the Transformer-PPO model across random, high-volatility, and low-volatility market windows.

The grouped maps show that the attention pattern remains uneven after averaging many test windows. This reduces the concern that the result comes only from one selected example. Across the grouped plots, the Transformer encoder still places more emphasis on some lookback positions than others.

The high-volatility and low-volatility groups show different temporal structures. In the high-volatility windows, the attention mass is distributed over a wider part of the lookback window. In the low-volatility windows, the pattern is somewhat more concentrated near recent positions. One possible interpretation is that unstable periods require the encoder to use information from a broader historical context, whereas calmer periods allow the representation to rely more on recent observations. This

interpretation should still be treated cautiously, since the attention maps provide diagnostic evidence rather than a causal explanation.

The difference map should not be read as a causal explanation of portfolio weights. It only shows where the two grouped attention patterns separate. The safer interpretation is that the encoder learns a structured temporal representation, and that this representation changes somewhat across volatility conditions.

5.5.3 Interpretation Scope and Statistical Support

The attention maps are not used as a direct explanation of the portfolio weights. They are included to inspect how the Transformer encoder distributes attention over the 20-day lookback window before the actor produces an allocation decision.

Appendix B reports an additional grouped analysis. The attention patterns are not uniform across all windows: high-volatility periods tend to show broader temporal attention, while more stable periods show a slightly stronger focus on recent observations. These results should be interpreted as evidence about the encoder’s temporal behavior, not as a causal explanation of the final trading action.

For this reason, the attention analysis is used only to support the explainability part of the thesis. The main empirical conclusions are still drawn from portfolio-level performance, risk, and turnover results.

5.6 Lookback Window Setting

The main experiments use a 20-day lookback window. This setting is used mainly to keep the state construction fixed and consistent across models. It is not meant to show that 20 days is the best possible horizon. In daily equity data, 20 trading days are close to one trading month, so this window gives the model a recent market context that is still straightforward to interpret.

There is also a practical reason for this choice. A very short window may make the policy respond too much to short-term noise, while a much longer window may bring in older observations that are less useful for the next allocation decision. For this reason, the 20-day window is used as a reasonable middle ground for the main

experiment.

The same lookback length is used across MLP-PPO, LSTM-PPO, and Transformer-PPO so that the learned-model comparison remains focused on encoder design rather than differences in state construction. Therefore, the role of the 20-day window in this thesis is methodological: it provides a fixed and practical input horizon for the main experiments. A more systematic search over lookback lengths is left for future work.

5.7 Integrated Discussion of Findings

The first research question can be answered positively, but only within the learned-model comparison. Transformer-PPO achieves the best cumulative return and Sharpe approximation among MLP-PPO, LSTM-PPO, and Transformer-PPO. Because the three learned models are tested with the same asset universe, lookback length, reward formulation, and PPO training setup, the comparison is mainly about the encoder. In that sense, the stronger result of Transformer-PPO can be linked to the way it constructs the market state.

This also explains why the result should not be overstated. The passive benchmarks remain strong in the main comparison, while the learned models are useful mainly because they provide adaptive, state-dependent allocation. Thus, the Transformer-PPO result supports the benefit of Transformer-based state encoding inside the learned PPO group, not a general claim that DRL dominates passive portfolio rules.

The benchmark result, however, makes the conclusion more cautious. Equal Weight and Buy-and-Hold Equal Weight still outperform all learned policies in the main five-year test period. Therefore, the thesis should not be read as evidence that DRL is generally better than passive allocation. A more accurate conclusion is that the Transformer encoder improves the learned PPO-based policy when compared with the selected MLP and LSTM baselines.

The answer to the second research question is also not one-sided. Adding the transaction-cost term changes the policy, but it does not improve every metric. In

the reported experiment, the cost-aware Transformer variant has a slightly higher cumulative return, while the no-cost variant performs better on several risk and trading-cost measures. This means that reward design should be judged through return, drawdown, turnover, and cost together, not only through final return.

The attention analysis adds supporting evidence for the explainability part of the thesis. The heatmaps and grouped statistics show that the Transformer encoder gives different levels of emphasis to different positions in the lookback window. The grouped results also suggest that this temporal pattern changes between high-volatility and low-volatility windows. These findings help describe the learned representation, but they should not be treated as causal explanations for individual portfolio weights.

These findings also provide some practical implications for portfolio management. First, the Transformer encoder may be more useful when market conditions are unstable, because the attention analysis indicates broader temporal emphasis in high-volatility windows. Second, the lookback window is not only a technical input choice; it can affect how reactive or conservative the learned policy becomes. A shorter window may make the policy more sensitive to recent noise, while a longer window may smooth the state representation but include less relevant older information. Third, reward design changes policy behavior, not only final return. The cost-aware reward leads to a different allocation profile and changes the return–risk–turnover trade-off, so learned portfolio strategies should be evaluated through both performance and rebalancing behavior.

Overall, the empirical results support the proposed framework in a controlled sense. The thesis shows how Transformer-based state encoding, PPO-based portfolio learning, transaction-cost-aware reward design, and temporal attention analysis can be combined in one dynamic allocation framework. Therefore, the contribution of this thesis is mainly methodological and empirical, rather than a claim that the proposed model is always superior in trading.

5.8 Chapter Summary

This chapter presented the empirical results of the proposed Transformer-PPO framework. The main benchmark comparison showed that Equal Weight and Buy-and-Hold Equal Weight remain very strong reference strategies in the five-year test period. At the same time, among the learned PPO-based models, Transformer-PPO achieved the strongest result.

The learned-model comparison therefore supports the role of Transformer-based state encoding, but only within the intended scope of the thesis. The result shows improvement over MLP-PPO and LSTM-PPO under the same PPO framework. It does not imply that the learned model dominates all passive allocation rules.

The reward-design experiment showed a mixed but meaningful effect. Transaction-cost-aware training changed the learned policy and slightly improved cumulative return, but the no-cost variant remained better on several risk and trading-cost measures. This supports the interpretation of reward design as a trade-off rather than a simple improvement mechanism.

Finally, the temporal attention analysis provided supporting interpretability evidence. The trained Transformer encoder showed structured emphasis across the lookback window, with attention patterns that differed across market conditions. These results help explain the representation behavior of the model, while the main conclusions remain based on portfolio-level performance, risk, turnover, and trading cost.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Chapter Overview

This chapter returns to the two main research questions and summarizes what can be learned from the empirical study. The conclusions in this chapter are placed within the scope of the study, as the current results are based on a set of five stocks, a fixed PPO framework, and a specific evaluation period.

First, this chapter summarizes the main findings from the empirical results. Then, it presents the remaining limitations of the study, including the small number of assets, the simplified trading environment, and the lack of constraints on the maximum weighting for each asset in the model. Finally, it suggests some possible directions for further research.

6.2 Summary of the Research

This thesis studied dynamic portfolio allocation as a sequential decision-making problem. Instead of forecasting the future price or direction of an individual stock, the proposed framework learns how portfolio weights should be adjusted over time based on recent multi-asset market information.

The framework combines a Transformer-based state encoder with a PPO actor–

critic policy. The Transformer encoder converts a rolling historical window into a latent market representation, while the actor network transforms this representation into long-only and fully invested portfolio weights. Transaction cost is incorporated into the reward function so that the policy is trained under a more realistic rebalancing objective rather than on gross return alone.

The empirical study was conducted on a controlled five-asset U.S. equity universe over the 2020–2024 period. The main learned-model comparison evaluated MLP-PPO, LSTM-PPO, and Transformer-PPO under the same PPO framework. A separate reward-design comparison examined cost-aware and no-cost Transformer variants. In addition, temporal attention maps were used as supporting interpretability evidence for the Transformer encoder. The 20-day lookback window was treated as a fixed state-construction setting for the main comparison, not as a separately optimized horizon.

6.3 Summary of Main Findings

The first result answers RQ1 at the level of the learned-model comparison. In the main five-year experiment, Transformer-PPO achieves higher cumulative return and Sharpe approximation than both MLP-PPO and LSTM-PPO. This suggests that the Transformer encoder provides a more useful market-state representation for the PPO policy than the two simpler encoder baselines used in this thesis.

However, this result should still be understood within the scope of the experiment. When the learned models are compared with all benchmark strategies, Equal Weight and Buy-and-Hold Equal Weight still perform better. Therefore, the thesis does not claim that DRL is generally superior to passive allocation. The more appropriate conclusion is narrower: under the same PPO framework, the Transformer encoder improves the learned, state-dependent allocation policy relative to MLP and LSTM encoders, while the passive strategies remain stronger in the full benchmark comparison.

The second result answers RQ2. Adding the transaction-cost penalty changes the learned Transformer policy, but the effect is mixed. The cost-aware variant produces

a slightly higher cumulative return under realistic evaluation. The no-cost variant, on the other hand, has a better Sharpe approximation, lower maximum drawdown, lower turnover, and lower trading cost. This means that the transaction-cost-aware reward affects the policy, but it does not improve every aspect of performance at the same time.

The attention analysis gives additional support to the explainability objective. The attention maps show that the Transformer encoder does not place equal emphasis on every position in the lookback window. Some historical positions receive more attention, and the grouped results suggest that the pattern changes between high-volatility and low-volatility windows. These results are useful for understanding the learned temporal representation. They should still be interpreted as diagnostic evidence, not as a direct causal explanation of the final portfolio weights.

6.4 Limitations

The results of this thesis need to be interpreted within the setting of the experiment. The main test uses only five large U.S. stocks: AAPL, MSFT, AMZN, JPM, and JNJ. Still, this is a small universe. It cannot reflect the full range of problems that would appear in a larger equity portfolio, an international portfolio, or a mixed-asset allocation setting with exchange-traded funds, bonds, commodities, and other instruments.

The backtest environment is also simplified. Transaction cost is already included in both the reward and the evaluation, which is important for this thesis, but other real trading frictions are still left out. The experiment does not include slippage, changing bid–ask spreads, liquidity limits, execution delay, taxation, or market impact. Because of that, the empirical results should be read as evidence from a controlled backtest, not as proof that the strategy can be used directly in live trading.

Another limitation is related to portfolio concentration. The softmax output keeps the portfolio long-only and fully invested, but it does not set a maximum weight for each stock. As a result, the learned policy can still place a large share of

capital in one asset. A more realistic version of the framework would need position limits, sector limits, or some form of diversification control.

A further limitation is the use of PPO as the only reinforcement learning optimizer. This choice keeps the learned-model comparison cleaner, because all encoder variants are trained under the same policy optimization method. At the same time, it also means that the conclusion is specific to the Transformer-PPO framework tested in this thesis. The study does not show whether the same encoder would lead to similar results under other actor-critic or policy-gradient methods.

The learned models are evaluated under a single fixed random seed to ensure reproducibility and fair comparison across encoder variants. While this setup maintains consistent experimental conditions, it does not fully assess sensitivity to random initialization and stochastic policy updates. Given the inherent stochasticity of DRL training, evaluating performance across multiple random seeds would provide a more rigorous assessment of policy stability and robustness.

6.5 Future Work

Future work can extend this thesis in several directions. The first direction is to test the framework on a larger asset universe. The current five-stock setting is useful for a controlled experiment, but it is still much simpler than a practical portfolio. Later studies could include more equities, sector-diversified portfolios, ETFs, international assets, bonds, commodities, or mixed-asset portfolios. This would help examine whether the Transformer encoder remains useful when the allocation problem becomes broader and more complex.

Another direction is to study the input window more carefully. This thesis uses a 20-day lookback window because it gives the model a recent and interpretable market context. However, this choice is not meant to be the optimal horizon. Future experiments could compare different lookback lengths and examine their effects on cumulative return, drawdown, turnover, and trading cost. This would make the state-construction choice more evidence-based.

The trading environment can also be improved. Although transaction cost is

included in this thesis, real trading involves more frictions. Future work could add slippage, time-varying bid–ask spreads, liquidity constraints, execution delay, and market impact.

Finally, later studies could refine both the portfolio constraints and the learning setup. Maximum-weight limits, sector exposure limits, or diversification penalties could be added to reduce concentration risk. Future work could also examine whether the Transformer-based state representation remains effective when combined with other deep reinforcement learning algorithms, such as SAC, TD3, or DDPG. In this thesis, PPO is fixed as the common optimizer to isolate the effect of the state encoder. Comparing such alternatives under the same reward function, transaction-cost setting, portfolio constraints, and evaluation protocol would help evaluate whether the findings are robust across different policy-optimization methods. In addition, future experiments could repeat the learned-model training under multiple random seeds and report mean and standard deviation. This would provide a clearer view of whether the observed differences come from the Transformer encoder, the optimizer, or training variability. Another possible extension is to evaluate the proposed framework under a walk-forward protocol to further examine its stability across changing market regimes. Reward design and interpretability can also be developed further, for example by testing drawdown-aware rewards or combining attention maps with perturbation tests and asset-level attribution.

6.6 Final Remarks

This study develops a Transformer-PPO framework for dynamic multi-asset portfolio allocation. The contribution of the thesis is not a new Transformer architecture or a new reinforcement learning algorithm. Rather, the main point is to place Transformer-based state encoding inside a controlled PPO allocation framework, together with transaction-cost-aware reward design and a limited attention-based interpretation layer.

The empirical results support a careful conclusion. In the main five-year experiment, Transformer-PPO performs better than MLP-PPO and LSTM-PPO among

the learned models. This finding suggests that Transformer-based state encoding can improve the market representation used by the learned PPO allocation policy relative to the selected encoder baselines. However, the result should not be read as evidence that DRL or Transformer-based trading dominates passive allocation. Equal Weight and Buy-and-Hold Equal Weight remain strong in the main benchmark comparison, so the learned policy is better understood as an adaptive, state-dependent allocation model rather than a universally superior trading strategy.

The reward-design and attention results should also be interpreted carefully. Adding a transaction-cost penalty changes the learned policy, but the improvement is not uniform across return, risk, turnover, and cost measures. Similarly, attention maps help inspect how the encoder uses the lookback window, but they do not fully explain the final portfolio weights. For this reason, the model should be evaluated through several dimensions at the same time: return, risk, drawdown, turnover, transaction cost, benchmark comparison, and the limited scope of interpretability. Within that scope, the thesis should be read as a controlled empirical study rather than as a claim of direct trading readiness.

Appendix A

SUPPLEMENTARY

PERFORMANCE RESULTS

This appendix provides supplementary figures for the empirical results discussed in Chapter 5. These figures are included as supporting material for the learned-model comparison and reward-design comparison.

A.1 Supplementary Learned-Model Metric Comparison

Figure A.1 provides a supplementary metric-level comparison of the learned PPO-based models. While Chapter 5 focuses on the main test table and the portfolio value curves, this figure gives an additional view of how MLP-PPO, LSTM-PPO, and Transformer-PPO differ across the reported portfolio metrics.

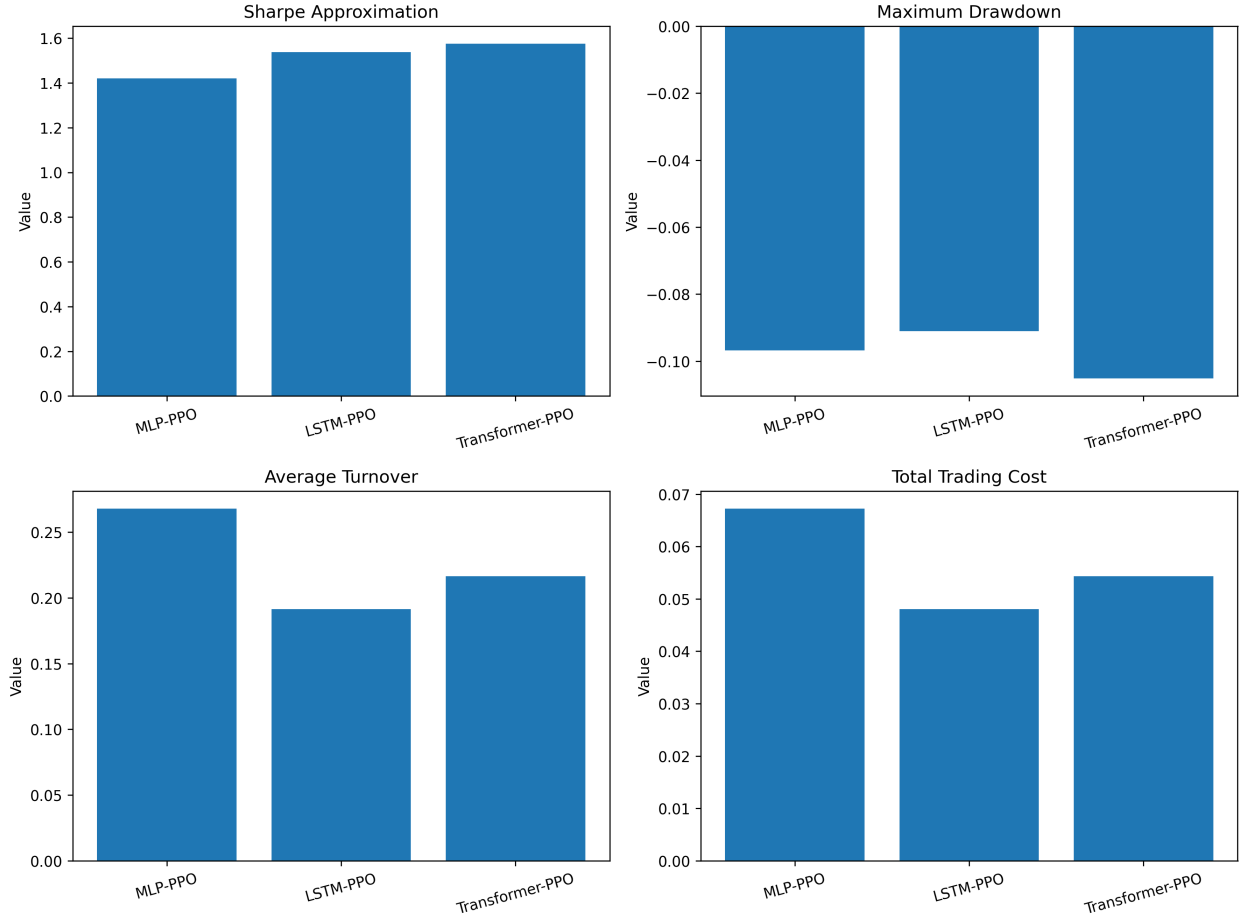


Figure A.1: Supplementary metric comparison of learned encoder variants in the five-year setting.

The supplementary comparison is consistent with the main learned-model results in Chapter 5. Transformer-PPO performs strongest on return-oriented learned-model metrics, while the risk and trading-cost metrics show that the result should not be interpreted as dominance across all dimensions.

A.2 Supplementary Reward-Design Comparison

Figure A.2 provides a supplementary visual summary of the reward-design comparison between the cost-aware-trained and no-cost-trained Transformer variants. This figure complements the main reward-design results in Chapter 5 by presenting the comparison across multiple metrics in a compact form.

Appendix A. SUPPLEMENTARY PERFORMANCE RESULTS

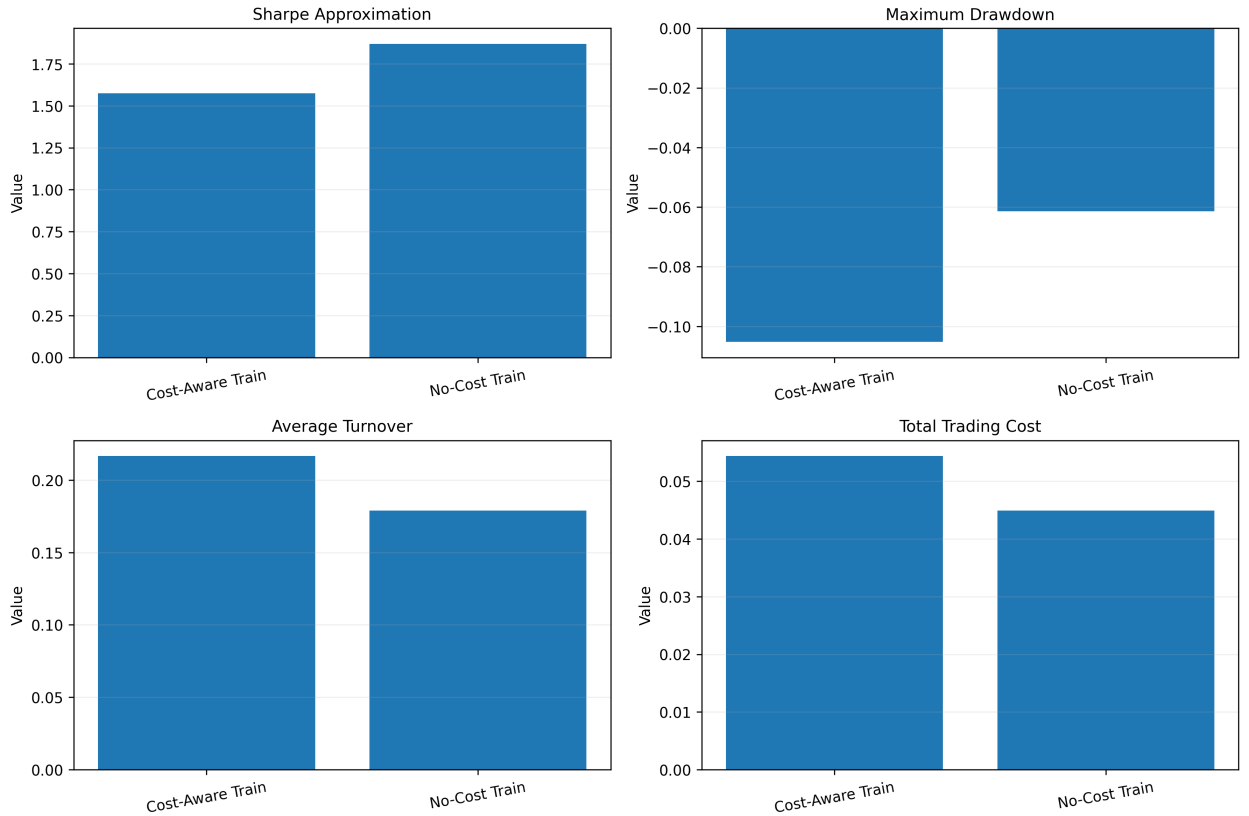


Figure A.2: Supplementary reward-design metric comparison for Transformer variants.

The supplementary reward-design comparison reinforces the main conclusion that transaction-cost-aware training changes the policy profile, but does not improve every metric. The result is best interpreted as a return–risk–turnover trade-off.

Appendix B

ADDITIONAL TEMPORAL ATTENTION ANALYSIS

This appendix provides additional material for the temporal attention analysis discussed in Chapter 5. The main chapter presents representative attention heatmaps and grouped attention maps to show how the trained Transformer encoder distributes attention across the historical lookback window. This appendix provides supplementary details, especially the grouped attention statistics used to support the interpretation of attention behavior across different market conditions.

The attention results are included as supporting interpretability evidence. They are not used to claim that attention weights fully explain portfolio decisions. Instead, they provide a diagnostic view of the temporal structure learned by the Transformer encoder before the actor network generates portfolio weights.

B.1 Grouped Attention Summary Statistics

Table B.1 reports grouped temporal attention summary statistics across different market-window groups. The reported groups include random test windows, high-volatility windows, and low-volatility windows. These statistics provide numerical support for the grouped attention interpretation discussed in Chapter 5.

Appendix B. ADDITIONAL TEMPORAL ATTENTION ANALYSIS

Table B.1: Grouped temporal attention summary statistics across market conditions.

Attention Metric	High-volatility	Low-volatility
Number of windows	30	30
Mean attention entropy	2.8644	2.8524
Std. of attention entropy	0.0449	0.0339
Mean recent-5 attention share	0.2484	0.2598
Std. of recent-5 attention share	0.0440	0.0550
Mean diagonal attention share	0.0387	0.0357
Std. of diagonal attention share	0.0041	0.0034
Mean attention center of mass	8.7982	9.0209
Std. of attention center of mass	0.7921	0.7945

The grouped statistics support the attention interpretation in Chapter 5. High-volatility windows show slightly higher attention entropy, while low-volatility windows show a somewhat stronger recent-window attention share. These patterns suggest that the Transformer encoder changes its temporal emphasis across market conditions, although the differences should be interpreted cautiously.

Table B.2 reports supplementary statistical comparisons between high-volatility and low-volatility attention metrics. These tests are included to check whether the grouped differences observed in the attention statistics are statistically strong or mainly directional.

Table B.2: Statistical comparison of temporal attention metrics between high-volatility and low-volatility windows.

Statistic	Mean entropy	Recent-5 share	Diagonal share	Recent center of mass
High-volatility mean	2.8644	0.2484	0.0387	8.7982
Low-volatility mean	2.8524	0.2598	0.0357	9.0209
Mean difference, high – low	0.0120	-0.0114	0.0030	-0.2227
Welch’s <i>t</i> -test <i>p</i> -value	0.2477	0.3798	0.0028	0.2814
Mann-Whitney <i>p</i> -value	0.1715	0.3478	0.0010	0.4464

The statistical comparison shows that not all grouped differences are equally strong. Therefore, the attention results are best described as structured and condition-sensitive tendencies, rather than fixed rules for each market condition.

B.2 Interpretation and Robustness Notes

The attention results should be interpreted as diagnostic evidence about the Transformer encoder, not as causal explanations of final portfolio weights. The grouped maps and statistics show that attention is structured and condition-sensitive, with some recent-focused tendency in relatively stable windows, but they do not establish direct asset-level attribution. Therefore, the attention analysis remains supplementary to the main portfolio-level evidence reported in Chapter 5.

Appendix C

EXTENDED-HORIZON ROBUSTNESS RESULTS

This appendix reports supplementary ten-year results to provide an extended-horizon robustness check for the main five-year experiment. The results are treated as supporting evidence only, while the core conclusions of the thesis remain based on the controlled five-year setting discussed in Chapter 5.

C.1 Ten-Year Experimental Setting

The ten-year setting follows the same general evaluation logic as the main five-year experiment, using the same asset universe, portfolio constraints, model families, and evaluation metrics where possible. It is included only as a supplementary robustness check.

C.2 Supplementary Ten-Year Result Comparison

Table C.1 reports the supplementary ten-year comparison of benchmark strategies and learned models. The table follows the same portfolio evaluation logic used in the main five-year results, including cumulative return, Sharpe approximation, maximum drawdown, turnover, and trading-cost-related metrics where available.

Appendix C. EXTENDED-HORIZON ROBUSTNESS RESULTS

Table C.1: Ten-year benchmark and learned-model performance comparison.

Model	Cumulative Return	Sharpe Approx.	Max Drawdown	Avg. Turnover	Total Trading Cost	Rank
BuyHold Equal Weight	0.7807	1.9359	-0.1136	0.0000	0.0000	1
Equal Weight	0.7242	1.9912	-0.1020	0.0000	0.0000	2
Transformer-PPO No-Cost Train	0.6451	1.9041	-0.1101	0.1467	0.0735	3
MLP-PPO 10Y	0.6363	2.2136	-0.1040	0.2279	0.1142	4
LSTM-PPO 10Y	0.6028	1.8763	-0.1077	0.1467	0.0735	5
Transformer-PPO Cost-Aware Train	0.5627	2.0576	-0.1237	0.2719	0.1362	6
Random Policy	0.2960	2.0132	-0.1580	0.6416	0.3214	7

The ten-year comparison again shows that passive benchmarks are difficult to beat. Buy-and-Hold Equal Weight and Equal Weight rank first and second by cumulative return. Among the learned models, the no-cost Transformer variant remains competitive, while the cost-aware Transformer variant is weaker in cumulative return. This appendix therefore suggests that learned DRL policies can be sensitive to the evaluation horizon and to the reward specification.

Figure C.1 shows the corresponding portfolio value paths for the ten-year setting. The curve plot is useful because it shows when the performance gaps appear, rather than only reporting the final values in the table.

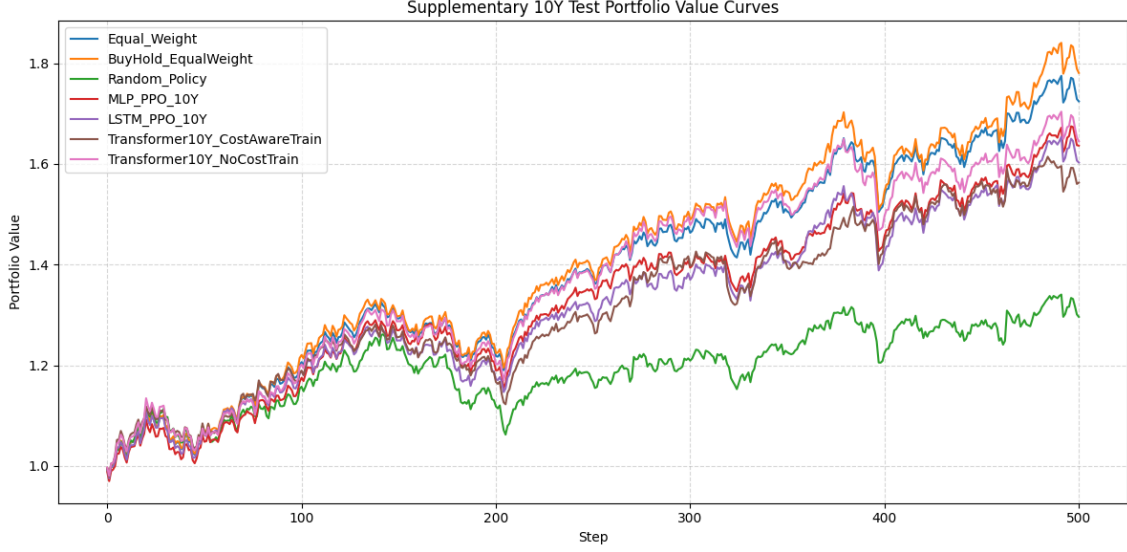


Figure C.1: Extended-horizon portfolio value curves under the ten-year setting.

The portfolio value curves are useful for identifying whether performance differences are stable throughout the extended period or concentrated in specific market segments. This is particularly important in financial reinforcement learning, where out-of-sample performance may depend strongly on regime changes, volatility conditions, and the timing of market drawdowns.

C.3 Interpretation Relative to the Main Five-Year Setting

The ten-year experiment is included as supplementary evidence only. It is useful for checking whether the main patterns remain visible over a longer period, but it should not replace the controlled five-year experiment used for the main conclusions.

The extended-horizon result is consistent with one message from Chapter 5: passive benchmarks remain strong references. The learned models remain competitive in several metrics, but their order is not stable once the evaluation horizon and reward setting change. For this reason, Appendix C should be treated as an additional robustness check. It supports a cautious reading of the results, rather than a claim that the proposed framework is superior in every market period.

Appendix D

IMPLEMENTATION NOTES

This appendix records the main implementation details used in the empirical experiments. It is included to make the experimental setup easier to trace, especially for the data construction choices, model settings, portfolio environment, and supplementary outputs.

The appendix should be read as supporting material for Chapter 4 and Chapter 5. The main methodological explanation remains in the body of the thesis, while this section provides a compact reference for reproducibility.

D.1 Feature Construction Details

The experiments use rolling-window states constructed from historical multi-asset market data. In the main setting, each state contains a 20-day lookback window for five assets: AAPL, MSFT, AMZN, JPM, and JNJ. The same feature-construction procedure is applied to MLP-PPO, LSTM-PPO, and Transformer-PPO, so the comparison mainly reflects encoder differences rather than preprocessing differences.

D.2 Hyperparameter Documentation

Table D.1 summarizes the main experimental configuration used in the empirical study. The table reports the settings that are most relevant for reproducing the main

five-year experiments.

Table D.1: Main experimental configuration

Component	Configuration
Asset universe	Five U.S. equities: AAPL, MSFT, AMZN, JPM, and JNJ
Data frequency	Daily observations
Data split	Training: 2020–2022; validation: 2023; testing: 2024
State input	20-day lookback window with shape $20 \times 5 \times F$, where F is the number of constructed features per asset
Portfolio constraint	Long-only and fully invested portfolio
Portfolio weights	Softmax-normalized allocation vector
Reward function	Portfolio log return with transaction cost penalty
Transaction cost rate	0.001
Learned models	MLP-PPO, LSTM-PPO, and Transformer-PPO
Transformer encoder	$d_{\text{model}} = 128$, 4 attention heads, 2 layers, FFN dimension = 256, dropout = 0.1
PPO optimizer	Learning rate = 3×10^{-4} , weight decay = 1×10^{-5}
PPO objective	Discount factor = 0.99, GAE $\lambda = 0.95$, clip range = 0.20
Training schedule	60 epochs, 10 PPO updates per epoch, mini-batch size = 128
Random seed	42

Additional implementation settings include value loss coefficient 0.5, entropy coefficient 0.001, maximum gradient norm 0.5, and Dirichlet concentration range from 5.0 to 200.0. These settings were kept fixed in the main five-year experiment.

D.3 Portfolio Environment and Evaluation Notes

The portfolio environment is long-only and fully invested. The actor network first produces asset scores, and these scores are converted into valid portfolio weights by the softmax layer. Transaction cost is computed from the change between two consecutive portfolio weight vectors. The strategies are then evaluated using cumulative return, Sharpe approximation, maximum drawdown, turnover, and trading cost.

D.4 Code Repository and Supplementary Materials

The code repository provides the notebooks and supplementary outputs used to reproduce the empirical workflow of this thesis.

Figure D.1 provides a QR code linking to the code repository and supplementary materials.



Figure D.1: QR code linking to the code repository and supplementary materials

Bibliography

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] J. Achiam, *Spinning Up in Deep Reinforcement Learning*, OpenAI, 2018. [Online]. Available: <https://spinningup.openai.com/en/latest/>
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv preprint arXiv:1707.06347, 2017.
- [4] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. Cambridge, U.K.: Cambridge Univ. Press, 2023.
- [5] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, Inc., 2017, pp. 5998–6008.
- [6] H. Markowitz, “Portfolio selection,” *J. Finance*, vol. 7, no. 1, pp. 77–91, Mar. 1952, doi: 10.1111/j.1540-6261.1952.tb01525.x.
- [7] Z. Jiang, D. Xu, and J. Liang, “A deep reinforcement learning framework for the financial portfolio management problem,” arXiv preprint arXiv:1706.10059, 2017.
- [8] Q. Y. E. Lim, Q. Cao, and C. Quek, “Dynamic portfolio rebalancing through reinforcement learning,” *Neural Comput. Appl.*, vol. 34, no. 9, pp. 7125–7139, 2022, doi: 10.1007/s00521-021-06853-3.

- [9] Y. Bai, Y. Gao, R. Wan, S. Zhang, and R. Song, “A review of reinforcement learning in financial applications,” *Annu. Rev. Stat. Appl.*, vol. 12, pp. 209–232, 2025, doi: 10.1146/annurev-statistics-112723-034423.
- [10] M. J. P. Magill and G. M. Constantinides, “Portfolio selection with transactions costs,” *J. Econ. Theory*, vol. 13, no. 2, pp. 245–263, Oct. 1976, doi: 10.1016/0022-0531(76)90018-1.
- [11] C. S. Pun and Z. Ye, “Optimal dynamic mean–variance portfolio subject to proportional transaction costs and no-shorting constraint,” *Automatica*, vol. 135, Art. no. 109986, 2022, doi: 10.1016/j.automatica.2021.109986.
- [12] J. Li, Y. Zhang, X. Yang, and L. Chen, “Online portfolio management via deep reinforcement learning with high-frequency data,” *Inf. Process. Manag.*, vol. 60, no. 3, Art. no. 103247, May 2023, doi: 10.1016/j.ipm.2022.103247.
- [13] C. Betancourt and W.-H. Chen, “Deep reinforcement learning for portfolio management of markets with a dynamic number of assets,” *Expert Syst. Appl.*, vol. 164, Art. no. 114002, 2021, doi: 10.1016/j.eswa.2020.114002.
- [14] T. W. Kim and M. Khushi, “Portfolio optimization with 2D relative-attentional gated transformer,” in *2020 IEEE Asia-Pacific Conf. Comput. Sci. Data Eng. (CSDE)*, Gold Coast, QLD, Australia, Dec. 16–18, 2020.
- [15] G. Song, T. Zhao, X. Ma, P. Lin, and C. Cui, “Reinforcement learning-based portfolio optimization with deterministic state transition,” *Inf. Sci.*, vol. 690, Art. no. 121538, Feb. 2025, doi: 10.1016/j.ins.2024.121538.
- [16] M.-E. Wu, J.-H. Syu, J. C.-W. Lin, and J.-M. Ho, “Portfolio management system in equity market neutral using reinforcement learning,” *Appl. Intell.*, vol. 51, no. 11, pp. 8119–8131, 2021, doi: 10.1007/s10489-021-02262-0.
- [17] Y. Jiang, J. Olmo, and M. Atwi, “Deep reinforcement learning for portfolio selection,” *Global Finance J.*, vol. 62, Art. no. 101016, Sep. 2024, doi: 10.1016/j.gfj.2024.101016.

- [18] P. Collin-Dufresne, K. D. Daniel, and M. Saglam, “Optimal dynamic asset allocation with transaction costs: The role of hedging demands,” *NBER Working Paper*, no. 33058, Oct. 2024.
- [19] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proc. 3rd Int. Conf. Learn. Representations (ICLR)*, 2015.
- [20] M.-T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” in *Proc. 2015 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Lisbon, Portugal, 2015, pp. 1412–1421, doi: 10.18653/v1/D15-1166.
- [21] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Trans. Neural Netw.*, vol. 12, no. 4, pp. 875–889, Jul. 2001.
- [22] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, “Deep direct reinforcement learning for financial signal representation and trading,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 3, pp. 653–664, Mar. 2017, doi: 10.1109/TNNLS.2016.2522401.
- [23] A. W. Lo, “The statistics of Sharpe ratios,” *Financial Analysts J.*, vol. 58, no. 4, pp. 36–52, 2002, doi: 10.2469/faj.v58.n4.2453.
- [24] D. H. Bailey and M. López de Prado, “The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality,” *J. Portfolio Manag.*, vol. 40, no. 5, pp. 94–107, 2014.
- [25] A. Rush, *The Annotated Transformer*, Harvard NLP, GitHub repository. [Online]. Available: <https://github.com/harvardnlp/annotated-transformer>
- [26] Yahoo Finance, *Yahoo Finance*. [Online]. Available: <https://finance.yahoo.com/>
- [27] A. Aroussi, *yfinance Documentation*. [Online]. Available: <https://ranaroussi.github.io/yfinance/>

- [28] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.